



ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ

ΣΧΟΛΗ ΤΕΧΝΟΛΟΓΙΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΕΠΙΚΟΙΝΩΝΙΩΝ

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

Πτυχιακή Εργασία

Τίτλος Πτυχιακής Εργασίας	Επιβίωση απέναντι στα zombies Survive the zombies
Ονοματεπώνυμο Φοιτητή	Θρασύβουλος Σκαρλής
Πατρώνυμο	Χρήστος
Αριθμός Μητρώου	Π15127
Επιβλέπων	Θεμιστοκλής Παναγιωτόπουλος, Καθηγητής

Ημερομηνία Παράδοσης Ιανουάριος 2025

Copyright ©

Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα. Οι απόψεις και τα συμπεράσματα που περιέχονται σε αυτό το έγγραφο εκφράζουν αποκλειστικά τον συγγραφέα και δεν αντιπροσωπεύουν τις επίσημες θέσεις του Πανεπιστημίου Πειραιώς. Ως συγγραφέας της παρούσας εργασίας δηλώνω πως η παρούσα εργασία δεν αποτελεί προϊόν λογοκλοπής και δεν περιέχει υλικό από μη αναφερόμενες πηγές.

Ευχαριστίες

Ευχαριστίες

Με την ολοκλήρωση της παρούσας πτυχιακής εργασίας, θα ήθελα να εκφράσω τις ειλικρινείς μου ευχαριστίες σε όλους εκείνους που με την υποστήριξή τους συνέβαλαν στην επιτυχή ολοκλήρωση αυτού του έργου.

Πρώτα και πάνω από όλα, θα ήθελα να ευχαριστήσω θερμά τον επιβλέποντα καθηγητή μου, κ. Παναγιωτόπουλο Θεμιστοκλή, ο οποίος ανέλαβε την επίβλεψη της πτυχιακής μου εργασίας και με καθοδήγησε καθ' όλη τη διάρκεια της διαδικασίας. Η συνεχής υποστήριξη, η πολύτιμη καθοδήγηση, καθώς και οι διορατικές του συμβουλές σε κάθε στάδιο της εργασίας, υπήρξαν εξαιρετικά σημαντικές και ανεκτίμητες. Χωρίς την καθοδήγησή του, δεν θα ήταν δυνατή η ολοκλήρωση αυτής της εργασίας στο παρόν επίπεδο. Η αφοσίωσή του στην επιστημονική έρευνα και η αμέριστη υποστήριξή του αποτέλεσαν για μένα πηγή έμπνευσης και δύναμης.

Επίσης, θα ήθελα να ευχαριστήσω όλα τα μέλη της επιτροπής αξιολόγησης για τον χρόνο και την προσοχή που αφιέρωσαν στην αξιολόγηση της εργασίας μου, καθώς και για τα εποικοδομητικά σχόλια και τις διορθώσεις τους, που συνέβαλαν στην περαιτέρω βελτίωσή της.

Θα ήθελα να εκφράσω τις θερμές μου ευχαριστίες στους καθηγητές και το προσωπικό του ΠΑΝΕΠΙΣΤΗΜΙΟΥ ΠΕΙΡΑΙΩΣ για τη συνεχή τους υποστήριξη κατά τη διάρκεια των σπουδών μου, την παροχή των απαραίτητων υλικών και την πρόσβαση σε επιστημονικές πηγές και πληροφορίες που συνέβαλαν στην έρευνά μου.

Επιπλέον, ευχαριστώ από καρδιάς τους φίλους μου για την ηθική τους υποστήριξη και τη βοήθεια σε δύσκολες στιγμές της πορείας μου. Η παρουσία και η συντροφιά τους ήταν για μένα πολύτιμη καθ' όλη τη διάρκεια της συγγραφής της εργασίας μου.

Τέλος, θα ήθελα να αφιερώσω αυτή την εργασία στους γονείς μου, που με αγάπη και αφοσίωση στήριξαν κάθε βήμα της εκπαιδευτικής μου πορείας. Η πίστη τους στις ικανότητές μου, η υπομονή τους και η αδιάκοπη ενθάρρυνσή τους μου έδωσαν τη δύναμη να ολοκληρώσω αυτό το έργο.

Με εκτίμηση και ειλικρινή ευγνωμοσύνη,

Σκαρλής Θρασύβουλος

Περίληψη

Η εργασία αυτή αφορά την ανάπτυξη ενός διαδραστικού βιντεοπαιχνιδιού που υλοποιήθηκε στο περιβάλλον του Unity3D, υλοποιημένη σε γλώσσα C# με τον τίτλο «Survive The Zombies». Το παιχνίδι θα είναι συμβατό με περιβάλλον Windows και σχεδιάστηκε με στόχο την ψυχαγωγία του χρήστη μέσω της συνεχούς πρόκλησής των ικανοτήτων του(που αφορά τη στόχευση) καθώς και την ενίσχυση της οξυδέρκειας του όσον αφορά το πώς να κινείται στο χώρο. Ο παίκτης αναλαμβάνει τον ρόλο του Τζακ, πρώην αναγνωρισμένος και έμπειρος στρατιώτης που είχε αποσυρθεί από το πεδίο μάχης και καλείται να υπερασπιστεί με τη ζωή του και το πάντα έμπιστο πιστόλι ως σύμμαχο μονάδα Βιολογικού Κινδύνου από τις στρατιές των zombies που έρχονται ακατάπαυστα.

Το παιχνίδι είναι ένα shooter survival/horde mode παιχνίδι, για το οποίο πηγή έμπνευσης ήταν τα παιχνίδια «Army of one», «call of duty/nazi zombies» και το «Left 4 dead».

Για την ανάπτυξη, χρησιμοποιήθηκαν εργαλεία όπως το C# για τον προγραμματισμό της λογικής του παιχνιδιού και το game engine Unity για τον σχεδιασμό του γραφικού περιβάλλοντος. Ειδικότερα, εφαρμόστηκαν τεχνικές [π.χ. φυσικής κίνησης, φωτισμού, διαχείρισης ήχου] και μηχανισμοί. αλληλεπίδρασης, συστήματος επιβράβευσης και συστήματα player progression . Η υλοποίηση επιδιώκει να προσφέρει μια ελκυστική και δυναμική εμπειρία χρήστη, με προκλήσεις που αυξάνονται σε δυσκολία όσο περισσότερη ώρα ο χρήστης επιβιώνει στο παιχνίδι.

Συνολικά, το παιχνίδι ανταποκρίνεται στους στόχους σχεδιασμού και παρέχει μια διασκεδαστική εμπειρία, επιδεικνύοντας τα τεχνικά και καλλιτεχνικά στοιχεία που αξιοποιήθηκαν κατά την ανάπτυξή του.

Λέξεις Κλειδιά: Unity3D,C#,game engine, Asset Store, Scripts, Animator, Game Development, 3D modelling

Abstract

This work involves the development of an interactive video game implemented in the Unity3D environment, using the C# language, titled "Survive The Zombies." The game is compatible with Windows and is designed to entertain the user by continuously challenging their skills (specifically targeting accuracy) while enhancing their agility in navigating the space. The player assumes the role of Jack, a former renowned and experienced soldier who has retired from the battlefield and is called upon to defend, with his life and his ever-trusted pistol as an ally, the Biohazard Containment Unit from the relentless waves of zombies.

The game is a shooter survival/horde mode, inspired by titles such as "Army of One," "Call of Duty/Nazi Zombies," and "Left 4 Dead."

For the development, tools like C# were used to program game logic, and the Unity game engine was utilized for the design of the graphical environment. Specifically, techniques were applied in [e.g., physics-based movement, lighting, sound management] and interaction mechanisms, reward systems, and player progression systems. The implementation seeks to deliver an engaging and dynamic user experience, with challenges that increase in difficulty the longer the player survives.

Overall, the game meets the design goals and offers an enjoyable experience, showcasing the technical and artistic elements utilized in its development.

Keywords: Unity3D, C#, game engine, Asset Store, Scripts, Animator, Game development, 3D modeling

Περιεχόμενα

Copyright ©	i
Ευχαριστίες	ii
Περίληψη	iii
ΛΕΞΕΙΣ ΚΕΛΕΙΔΙΑ.....	iii
Abstract.....	iii
KEY WORDS.....	iii
Πίνακας Περιεχομένων.....	iv
1 Εισαγωγή	1
1.1 Περίληψη	2
1.2 Κίνητρο για την Διεξαγωγή της Εργασίας	3
1.3 Σκοπός και στόχοι της Εργασίας	3
1.4 Δομή της Εργασίας	3
2 Υπόθεση	3-5
3 Overview παιχνιδιού	5-7
3.1 Είδος: Horde Mode	5
3.2. Μηχανισμοί Παίκτη	6-7
3.3 Σύστημα Εμπειρίας (XP)	8
3.4. Τύποι και Ιδιότητες Ζόμπι	9-12
3.5. Συμβάν "Loud Scream": Μηχανισμός Ενίσχυσης Ζόμπι	12
4 Μαθηματικά του Παιχνιδιού	12-14
5 Scenes	15-18
5.1 Οι σκηνές/Scenes στο Unity	15
5.2 Τα Scenes του παιχνιδιού	15-18
6 Animation/Animator	19-29
6.1 Βασικά Στοιχεία του Animator	19
6.2 Πως Λειτουργεί το Animator	19
6.3 Χαρακτηριστικά του Animator	19
6.4 Animation Rigging	20
6.5 Πλεονεκτήματα του Animation Rigging	20
6.6 Animator και Animation Rigging στο "Survive The Zombies"	21-29
7Scripts.....	30-44
7.1 Εισαγωγή στα Scripts.....	30
7.2 Βασικά Scripts των zombies	30-37
7.3 Βασικά Scripts του παίκτη	37-44
8 Μελλοντικά Implementations	44-45
9 Ορολογία	45-47
10 Συμπεράσματα	47

1 Εισαγωγή

Η πτυχιακή εργασία αποτελεί μια εξαιρετική ευκαιρία για τον φοιτητή να εμβαθύνει σε θέματα που τον ενδιαφέρουν και μπορούν να συμβάλουν ουσιαστικά στην επαγγελματική του αποκατάσταση. Παράλληλα, λειτουργεί ως προσομοίωση της εργασιακής διαδικασίας, καθώς περιλαμβάνει όλα τα στάδια ανάπτυξης ενός έργου, βοηθώντας τον να κατανοήσει τα διαδοχικά βήματα που απαιτούνται για την ολοκλήρωσή του. Στην παρούσα πτυχιακή, η υλοποίηση επικεντρώθηκε στην κατανόηση των διαδικασιών δημιουργίας ενός ψηφιακού παιχνιδιού.

Συγκεκριμένα, στο πλαίσιο του game development, ο φοιτητής εξοικειώνεται με τεχνικές σχεδιασμού, ανάπτυξης και δοκιμών, αξιοποιώντας εργαλεία προγραμματισμού και λογισμικά που χρησιμοποιούνται ευρέως στη βιομηχανία των παιχνιδιών. Αυτή η διαδικασία τον προετοιμάζει να αντιμετωπίσει τις πραγματικές προκλήσεις της παραγωγής και τον βοηθά να κατανοήσει τις απαιτήσεις ενός ολοκληρωμένου παιχνιδιού – από τον αρχικό σχεδιασμό και τη δημιουργία γραφικών, μέχρι την κωδικοποίηση και τον τελικό έλεγχο ποιότητας.

Ιστορική Αναδρομή στο Game Development

Η ιστορία του game development ξεκίνησε τη δεκαετία του 1950, όταν δημιουργήθηκαν τα πρώτα βιντεοπαιχνίδια. Ένα από τα πρώτα παραδείγματα είναι το παιχνίδι "Tennis for Two" (1958), που αναπτύχθηκε από τον φυσικό William Higinbotham. Ακολούθησε το διάσημο "Spacewar!" (1962) από τον Steve Russell, που θεωρείται το πρώτο ηλεκτρονικό παιχνίδι που προγραμματίστηκε σε έναν υπολογιστή και προκάλεσε μεγάλο ενδιαφέρον για την ανάπτυξη της βιομηχανίας.

Η δεκαετία του 1970 σηματοδότησε την εμφάνιση των πρώτων μεγάλων εταιριών παραγωγής βιντεοπαιχνιδιών. Η Atari ήταν από τις πρωτοπόρες, κυκλοφορώντας το κλασικό "Pong" (1972), το οποίο άνοιξε τον δρόμο για την ανάπτυξη arcade παιχνιδιών. Ακολούθησαν άλλες εταιρείες, όπως η Nintendo και η Sega, που σταδιακά ανέπτυξαν την παραγωγή ηλεκτρονικών παιχνιδιών σε οικιακές κονσόλες. Η εισαγωγή της Nintendo Entertainment System (NES) το 1983 έφερε τα βιντεοπαιχνίδια στα σπίτια εκατομμυρίων ανθρώπων, καθιστώντας την ενασχόληση με τα ηλεκτρονικά παιχνίδια πιο σύνηθες.

Τα πρώτα game engine (μηχανές ανάπτυξης παιχνιδιών) έκαναν την εμφάνισή τους στις αρχές της δεκαετίας του 1990. Η id Software ανέπτυξε τον πρώτο μηχανισμό 3D rendering για παιχνίδια, το Doom Engine, το οποίο χρησιμοποιήθηκε για το θρυλικό παιχνίδι Doom (1993) και θεωρείται το πρώτο σημαντικό βήμα για τη δημιουργία σύγχρονων game engines. Μετά το Doom, κυκλοφόρησαν κι άλλα εξειδικευμένα game engines, όπως το Unreal Engine της Epic Games το 1998, που παραμένει ακόμα και σήμερα από τις πιο δημοφιλείς πλατφόρμες ανάπτυξης παιχνιδιών.

Ιστορική Αναφορά στο Unity.

Το Unity δημιουργήθηκε το 2005 από τους David Helgason, Nicholas Francis και Joachim Ante, τρεις προγραμματιστές με όραμα να δημιουργήσουν ένα ευέλικτο και προσιτό εργαλείο ανάπτυξης παιχνιδιών. Με την κυκλοφορία του, η Unity αποτέλεσε μια πρωτοποριακή επιλογή για τους ανεξάρτητους προγραμματιστές (indie developers), καθώς ήταν το πρώτο game engine που προσέφερε φιλικό περιβάλλον εργασίας και προσιτές τιμές. Έτσι, έδωσε τη δυνατότητα σε μικρές ομάδες ή μεμονωμένους δημιουργούς να αναπτύσσουν παιχνίδια χωρίς να χρειάζονται μεγάλους προϋπολογισμούς και πόρους.

Αρχικά, η Unity ήταν διαθέσιμη μόνο για macOS, αλλά με την πάροδο του χρόνου εξελίχθηκε για να υποστηρίζει πολλαπλές πλατφόρμες, όπως Windows, Android, iOS και πολλές κονσόλες. Αυτό το χαρακτηριστικό την κατέστησε εξαιρετικά δημοφιλή, καθώς οι developers μπορούσαν να δημιουργήσουν ένα παιχνίδι και να το δημοσιεύσουν εύκολα σε πολλαπλές πλατφόρμες χωρίς να χρειάζεται να αναπτύξουν ξανά από την αρχή.

Περιγραφή του Unity Game Engine

Το Unity είναι ένα σύγχρονο, ολοκληρωμένο game engine που επιτρέπει στους προγραμματιστές να δημιουργούν 2D, 3D και VR/AR (virtual και augmented reality) παιχνίδια. Διαθέτει ένα εύχρηστο περιβάλλον εργασίας με γραφικά εργαλεία, δυνατότητες scripting μέσω της γλώσσας προγραμματισμού C#, και μια μεγάλη βιβλιοθήκη από resources και plugins που υποστηρίζουν την ταχεία ανάπτυξη εφαρμογών.

Τα βασικά χαρακτηριστικά του Unity περιλαμβάνουν:

Εργαλεία για 2D και 3D γραφικά: Τα εργαλεία αυτά επιτρέπουν στους δημιουργούς να χτίζουν τρισδιάστατα και δισδιάστατα περιβάλλοντα, χαρακτήρες και αντικείμενα με λεπτομέρεια και ακρίβεια.

- Physics Engine: Παρέχει ρεαλιστικές φυσικές αλληλεπιδράσεις, κάνοντας τη συμπεριφορά των αντικειμένων στο παιχνίδι πιο φυσική.
- Εξαγωγή σε Πολλαπλές Πλατφόρμες: Το Unity επιτρέπει τη δημιουργία ενός έργου που μπορεί εύκολα να εξαγάγει και να προσαρμόσει σε διάφορες πλατφόρμες, συμπεριλαμβανομένων των Windows, MacOS, iOS, Android, και κόνσολών όπως PS και Xbox.
- Asset Store: Ένα marketplace που παρέχει πρόσβαση σε χιλιάδες assets, όπως γραφικά, ήχους και κώδικα, που μπορούν να ενσωματωθούν εύκολα στο έργο.
- Συμβατότητα με VR/AR: Υποστηρίζει την ανάπτυξη παιχνιδιών και εφαρμογών σε εικονική και επαυξημένη πραγματικότητα, κάτι που είναι πλέον σημαντικό στην αγορά των βιντεοπαιχνιδιών.

Σήμερα, το Unity συνεχίζει να εξελίσσεται, ενσωματώνοντας τεχνολογίες αιχμής και παρέχοντας καινοτόμα εργαλεία για developers όλων των επιπέδων.

1.1 Περίληψη

Σκοπός της πτυχιακής αυτής ήταν η δημιουργία ενός τρισδιάστατου παιχνιδιού πρώτου προσώπου με κυρίαρχο σκοπό να δοκιμάσουμε τις μηχανικές δεξιότητες του χρήστη, το στόχο του καθώς επίσης και τη ικανότητά του να κινείται αποδοτικά στον χώρο και να χρησιμοποιεί αυτόν προς όφελός του για να αποφύγει τους εχθρούς.

Ταυτόχρονα όμως θέλαμε να κάνουμε τον χρήστη να ταυτιστεί με τον πρωταγωνιστή μας και να του δημιουργηθεί ένα αίσθημα πως ο κόπος του ανταμείβεται και όσο περισσότερο χρόνο περνάει στο παιχνίδι τόσο δυνατότερος θα γίνεται.

Αυτό επιτευχθεί δίνοντας στον παίκτη μας στατιστικά (stats) που δυναμώνουν όσο παίρνει εμπειρία (XP) σκοτώνοντας zombie. Αυτό δημιουργεί ένα κίνητρο ώστε ο χρήστης να μείνει προσηλωμένος στο στόχο του που είναι φυσικά να επιβιώσει όσο το δυνατόν περισσότερο.

Για την απόκτηση των μοντέλων που χρησιμοποιήθηκαν αποκλείστηκαν μοντέλα που αποκτήθηκαν μέσω του asset store του Unity.

Ενώ για τα animations χρησιμοποιήθηκε ένα εργαλείο που παρέχει το ίδιο το Unity το οποίο αποκαλεί Animator καθώς και το Mixamo μια πλατφόρμα της Adobe, που προσφέρει έτοιμα 3D μοντέλα χαρακτήρων και animations, παρέχοντας στους χρήστες μια βιβλιοθήκη από προκατασκευασμένες κινήσεις και μοντέλα.

1.2 Κίνητρο για την Διεξαγωγή της Εργασίας

Ένα από τα βασικά κίνητρα για τη δημιουργία αυτής της πτυχιακής εργασίας ήταν η μεγάλη μου αγάπη για τα video games, στα οποία έχω αφιερώσει αμέτρητες ώρες από τότε που ήμουν παιδί μέχρι και σήμερα. Ήταν, λοιπόν, φυσικό να θελήσω να φτιάξω κάποια στιγμή ένα

δικό μου παιχνίδι. Επέλεξα να ασχοληθώ με μια πτυχιακή εργασία στον τομέα του game development, καθώς θεώρησα ότι είναι μια ιδανική ευκαιρία για να κατανοήσω από πρώτο χέρι τις διαδικασίες που απαιτούνται για την ανάπτυξη ενός ηλεκτρονικού παιχνιδιού και να δω πώς μπορώ να τις αντιμετωπίσω αποτελεσματικά.

1.3 Σκοπός και στόχοι της Εργασίας

Ο στόχος της πτυχιακής μας, εκτός από το τελικό αποτέλεσμα που ήθελα να επιτύχω, ήταν να αναπτύξω τις δεξιότητές μου σε όλα τα στάδια της δημιουργίας ενός ηλεκτρονικού παιχνιδιού. Αυτά τα στάδια περιλαμβάνουν πολλές και πολύπλοκες διαδικασίες, όπως το 3D modelling, το animation, το level designing και το scripting, τα οποία απαιτούν εξειδίκευση.

Οι στόχοι αυτοί ήταν βασικοί, και έπρεπε, σε κάποιο βαθμό, να ανταποκριθώ στις προκλήσεις που θα προέκυπταν. Άλλωστε, σε μια μεγάλη παραγωγή, κάθε ένας από αυτούς τους τομείς ανατίθεται σε εξειδικευμένες ομάδες με σημαντική εμπειρία και εξειδίκευση, καθώς θεωρούνται διακριτοί και αυτόνομοι ρόλοι.

1.4 Δομή της Εργασίας

Η δομή της εργασίας παρουσιάζεται από κάτω:

- Στο πρώτο κεφάλαιο παρουσιάζεται η περίληψη της εργασίας που αναλύονται με λίγα λόγια τόσο τα κίνητρα όσο και ο σκοπός της εργασίας.
- Στο δεύτερο κεφάλαιο θα παρουσιαστεί η Ιστορία και το backstory του πρωταγωνιστή της ιστορίας.
- Στο τρίτο κεφάλαιο γίνεται ένα overview του παιχνιδιού, όπου αναλύονται και τα mechanics του παιχνιδιού.
- Στο τέταρτο κεφάλαιο θα εξηγηθούν τα μαθηματικά που κρύβονται πίσω από τις όποιες λειτουργίες του παιχνιδιού.
- Στο πέμπτο κεφάλαιο παρατίθενται τα 3 διαφορετικά scenes του παιχνιδιού.
- Στο έκτο κεφάλαιο γίνεται αναφορά στους τομείς των Animations και Animator.
- Στο έβδομο κεφάλαιο αναλύονται ορισμένα scripts του παιχνιδιού
- Το πραγματεύεται μελλοντικές εφαρμογές και implementations που μπορούν να προστεθούν στο μέλλον.
- Το ένατο κεφάλαιο περιέχει την ορολογία που χρησιμοποιείται στην εργασία.
- Τέλος στο δέκατο και τελευταίο κεφάλαιο γίνεται αναφορά στις τελικές παρατηρήσεις.

2 Υπόθεση

Ιστορία/Story

Η έρημος εκτεινόταν απέραντη κάτω από έναν καυτό ήλιο, άδεια και αμείλικτη, διακοπτόμενη μόνο από το σκοτεινό περίγραμμα του εγκαταλελειμμένου εργοστασίου χάλυβα. Σκουριασμένο και ερειπωμένο, δεσπόζει στο τοπίο σαν ένα ξεχασμένο απομεινάρι από άλλη εποχή. Το εργοστάσιο, κάποτε ένας πολυσύχναστος κόμβος μηχανημάτων και εργατών, είναι τώρα σιωπηλό, διακοπτόμενο μόνο από το απαλό σφύριγμα του ανέμου που περνά μέσα από σπασμένα παράθυρα και στριμμένα δοκάρια. Ο Τζακ Χέις στέκεται μόνος στις σκιές αυτού του ερημικού μέρους, με τα μάτια του να σαρώνουν τον μακρινό ορίζοντα.

Το πρόσωπο του Τζακ είναι σημαδεμένο από την κούραση, με ταλαιπωρημένα μάτια και ένα περίστροφο στο χέρι του. Ένας μικρός πομποδέκτης στο ζωνάρι του βγάζει έναν απαλό θόρυβο, η σιωπή του ίση με το κενό γύρω του. Με κάθε λεπτό που περνά, νιώθει το βάρος της αποστολής του να τον πιέζει – ένα καθήκον που ξεπερνά τη δική του επιβίωση.

Όλα άρχισαν με μία μόλυνση – ένα παθογόνο μικρόβιο που είχε αρχικά σχεδιαστεί ως όπλο, προκειμένου να αποδυναμώνει τους εχθρούς στον πόλεμο. Γνωστό ως Παθογόνο B7, ο ιός είχε δημιουργηθεί σε μυστικό επίπεδο και είχε σχεδιαστεί να αποδυναμώνει αντί να σκοτώνει. Όμως, κάτι πήγε τραγικά λάθος. Ένα εργαστηριακό λάθος τον μετάλλαξε, εξαπλώνοντας τη μόλυνση ανεξέλεγκτα, μετατρέποντας τους ανθρώπους σε αηδιαστικά, ανθρωποφάγα πλάσματα χωρίς ίχνος ανθρωπιάς. Ολόκληρες πόλεις κατέρρευσαν μέσα σε λίγες μόνο εβδομάδες. Ο κόσμος πάλευε, εξεγείρονταν και τελικά υπέκυπτε στην αμείλικτη εξάπλωση της μόλυνσης.

Μέσα στο χάος, οι άνθρωποι στην εξουσία κλείστηκαν σε οχυρωμένα καταφύγια, στέλνοντας μια αγωνιώδη κλήση στα λίγα εναπομείναντα στρατεύματα και παλιούς συνεργάτες που μπορούσαν ακόμη να εμπιστευτούν. Μερικές στρατηγικές τοποθεσίες χαρακτηρίστηκαν ως κρίσιμες για προστασία – τοποθεσίες που έκρυβαν την τελευταία ελπίδα για τη δημιουργία θεραπείας. Το εργοστάσιο χάλυβα που φύλαγε ο Τζακ ήταν ένα από αυτά. Κάτω από το σκουριασμένο δάπεδο και το φθαρμένο μπετόν του φυλάσσονταν κάτι πολύτιμο: δείγματα του Παθογόνου B7 και αρχικά στάδια ενός πειραματικού αντιτοξίνου. Ήταν απομεινάρια από μια εποχή που οι επιστήμονες πίστευαν ότι μπορούσαν να ελέγξουν τη μόλυνση και, αν ανακτηθούν, αυτά τα δείγματα ίσως να περιέχουν το κλειδί για τη δημιουργία αντίδοτου.

Ο Τζακ, πρώην αναγνωρισμένος και έμπειρος στρατιώτης που είχε αποσυρθεί από το πεδίο μάχης, είχε εγκαταλείψει τις μέρες του στον στρατό μετά από μια επώδυνη προδοσία στην τελευταία του αποστολή. Η δουλειά στο εργοστάσιο ήταν τέλεια – απομονωμένη, μοναχική και χωρίς περιπέτειες. Αλλά όλα άλλαξαν όταν έλαβε ένα μήνυμα από τον Λοχία Μίλερ, τον παλιό του συνάδελφο και πιο στενό φίλο. Ο Μίλερ ήταν η τελευταία σύνδεση του Τζακ με την παλιά του ζωή, ένας αδερφός στα όπλα που του είχε σώσει τη ζωή παραπάνω από μία φορά.

«Τζακ, είμαι ο Μίλερ. Ξέρω ότι έχεις αποσυρθεί, αλλά σε χρειάζομαι για μια τελευταία αποστολή. Συναντήσου μαζί μου στο Steelworks έξω στην έρημο. Το μέρος που μιλήσαμε... Δεν είναι μόνο ένα εργοστάσιο. Έχουμε μια ευκαιρία να διορθώσουμε τα πράγματα.»

Ο Τζακ πήγε χωρίς δισταγμό, με την αφοσίωσή του αμετάβλητη. Ο Μίλερ εξήγησε ότι το εργοστάσιο ήταν κάτι παραπάνω από ένα απομεινάρι – ήταν ένα οχυρό που έκρυβε στρατιωτικά αποθέματα και ένα θησαυροφυλάκιο με βιολογικά δείγματα. Αυτά τα δείγματα, επέμενε ο Μίλερ, ίσως να ήταν η τελευταία ελπίδα της ανθρωπότητας. Αν μπορούσε να αντιγραφεί το αντιτοξίνο, η μόλυνση ίσως να μπορούσε να σταματήσει ή ακόμη και να θεραπευτεί.

Μαζί, ο Τζακ και ο Μίλερ οχύρωσαν το εργοστάσιο, σφραγίζοντας τις εισόδους και τοποθετώντας παγίδες όπου μπορούσαν. Καθώς οι μέρες περνούσαν, οι ζωντανοί-νεκροί συγκεντρώνονταν γύρω από το εργοστάσιο, με επιμονή και απειλή. Ο Τζακ και ο Μίλερ πολέμησαν πλάι-πλάι, απωθώντας το ένα κύμα μετά το άλλο, μέχρι που μια μάχη αποδείχθηκε μοιραία για τον Μίλερ. Κατά τη διάρκεια μιας απελπισμένης συμπλοκής, δέχθηκε δάγκωμα, και ο Τζακ είδε τον φίλο του να υποκύπτει στη μόλυνση, αναγκασμένος να δει τον άνθρωπο που κάποτε αποκαλούσε αδερφό να μετατρέπεται σε ένα στρεβλωμένο, χωρίς συνείδηση κέλυφος.

Πριν «μεταμορφωθεί», ο Μίλερ έκανε τον Τζακ να υποσχεθεί ότι θα κρατούσε το μέρος ασφαλές. Τον κοίταξε με μάτια θολά αλλά αποφασισμένα, λέγοντάς του, «Μην τους αφήσεις να μπου, Τζακ. Μην αφήσεις να πάει χαμένο.»

Τώρα ο Τζακ στεκόταν μόνος του, φρουρώντας τη μονάδα Βιολογικού Κινδύνου όπου φυλάσσονταν τα δείγματα. Οι οχυρώσεις του κρατούσαν, αλλά τα αποθέματα του τελείωναν, και τα πυρομαχικά του ήταν λιγοστά. Ο πομποδέκτης ήταν σιωπηλός, αν και συνέχιζε να ελπίζει ότι κάποια στιγμή θα κατέφθαναν οι δυνάμεις εξαγωγής, ότι κάποιος ίσως να ερχόταν για να πάρει τα δείγματα που ο Μίλερ έδωσε τη ζωή του για να προστατεύσει. Όμως, όσο οι μέρες περνούσαν, αυτή η ελπίδα άρχισε να ξεθωριάζει.

Ένας χαμηλός βρυχηθμός διέκοψε τη σιωπή, τραβώντας την προσοχή του Τζακ στην άκρη του εργοστασίου. Σκιές άρχισαν να εμφανίζονται πάνω από τους αμμόλοφους, σκιές που πολλαπλασιάζονταν, συγκεντρώνοντας σαν ένα σκοτεινό κύμα. Προχωρούσαν μπροστά, διεστραμμένες μορφές που πλησίαζαν το εργοστάσιο σε αργά, αδιάκοπα κύματα.

Ο Τζακ έσφιξε το πιστόλι του και μίλησε στον πομποδέκτη του, η φωνή του τραχιά και κουρασμένη.

«Εδώ ο Τζακ Χέις... ακόμα περιμένω στο Steelworks. Τα δείγματα του αντιδότου είναι εδώ, επαναλαμβάνω, έχουμε τα δείγματα του αντιδότου. Απλά... στείλτε κάποιον. Θα τους κρατήσω απ' έξω.»

Πήρε μια σταθερή ανάσα και κοίταξε τα φθαρμένα διακριτικά γύρω από τον καρπό του – τα διακριτικά του Μίλερ, το τελευταίο ενθύμιο της θυσίας του φίλου του. Ήταν αποφασισμένος να μην αφήσει τον θάνατο του Μίλερ να πάει χαμένος. Θα κρατούσε τη θέση του, θα πολεμούσε τα κύματα των ζωντανών-νεκρών όσο μπορούσε και θα κρατούσε τα δείγματα ασφαλή για όσο μπορούσε.

Καθώς οι πρώτοι μολυσμένοι έφτασαν στην εξωτερική περίφραξη, ο Τζακ σήκωσε το πιστόλι του, ακλόνητος. Η ορδή θα ερχόταν, ατελείωτη και αδηφάγα, αλλά ο Τζακ Χέις δεν θα παραδιδόταν. Όχι όσο υπήρχε μια πιθανότητα, όσο μικρή κι αν ήταν, ότι η στάση του θα μπορούσε να κάνει τη διαφορά σε έναν κόσμο που είχε βυθιστεί στο σκοτάδι.

Και έτσι ετοιμάστηκε, έτοιμος να κάνει την τελευταία του μάχη στο Steelworks.

Backstory/Υπόβαθρο Τζακ Χέις

Ο Τζακ Χέις ήταν πάντα ένας προστάτης. Σαν ένας έμπειρος αναγνωριστικός στρατιώτης στον στρατό, ήταν γνωστός για την ψυχραιμία του κάτω από πίεση και την αφοσίωσή του στην ομάδα του. Αλλά όταν η τελευταία του αποστολή τελείωσε με προδοσία και απώλεια, ο Τζακ αποφάσισε ότι είχε δει αρκετά από την ασχήμια του κόσμου. Άφησε τη θέση του και αποσύρθηκε σε μια ήσυχη ζωή, παίρνοντας δουλειά ως νυχτοφύλακας σε ένα απομονωμένο εργοστάσιο στην έρημο. Το εργοστάσιο χάλυβα ήταν το καταφύγιό του, ένας τόπος όπου επιτέλους μπορούσε να βρει λίγη γαλήνη.

Ωστόσο, η αφοσίωσή του δεν εξασθένησε ποτέ. Όταν ο Μίλερ, η τελευταία του σύνδεση με εκείνη την παλιά ζωή, τον κάλεσε για βοήθεια, ο Τζακ δεν μπορούσε να αρνηθεί. Ο Μίλερ ήταν ο αδερφός που ποτέ δεν είχε, ο άνθρωπος που του είχε σώσει τη ζωή κατά τη διάρκεια μιας ενέδρας πριν από χρόνια.

3 Overview παιχνιδιού

3.1 Είδος: Horde Mode

Στόχος: Ο παίκτης ξεκινάει από το level1/Lv1 (επίπεδο 1) με περιορισμένα Ammo (πυρομαχικά) και πρέπει να επιβιώσει από κύματα ζόμπι σκοτώνοντάς τα, κερδίζοντας experience/XP (πόντους εμπειρίας) και ανεβαίνοντας επίπεδο. Κάθε level δίνει στον παίκτη αυξημένα στατιστικά (stats) και βελτιώνει τις πιθανότητες επιβίωσης ενάντια σε ολοένα πιο δύσκολους τύπους ζόμπι. Τελικός στόχος είναι να επιβιώσεις όσο δυνατών περισσότερη ώρα γίνεται



Kill Count

3.2. Μηχανισμοί Παίκτη

Αρχικά Στατιστικά Παίκτη – Player Stats:

Speed (Ταχύτητα): 4

Sprint Multiplier (Πολλαπλασιαστής Σπριντ): x1.2

Reload Speed (Χρόνος Επαναγέμισης): 4 δευτερόλεπτα

Ammo/Max Ammo (Πυρομαχικά/Μέγιστα Πυρομαχικά): 18

BulletDamage (Ζημιά Σφαίρας): 15



Level παίχτη

Μηχανισμοί Ανεβάσματος Επιπέδου- Level-Up Mechaics:

Speed Increase(Αύξηση Ταχύτητας) ανά Επίπεδο: +0.01

Sprint Multiplier Increase(Αύξηση Πολλαπλασιαστή Σπριντ) ανά Επίπεδο: +0.01

Reload Time decrease(Μείωση Χρόνου Επαναγέμισης) ανά Επίπεδο: -0.03 δευτερόλεπτα

Ammo/Max Ammo Increase(Αύξηση Πυρομαχικών/Μέγιστων Πυρομαχικών) ανά Επίπεδο: +1

Μηχανισμοί Υγείας- Health Mechanics:

Levels (Επίπεδα) 1-20:

Flat Heal (Σταθερή Θεραπεία) ανά Level (Επίπεδο): +3

Max Health Increase (Αύξηση Μέγιστης Υγείας) ανά Επίπεδο: +3

Levels (Επίπεδα) 21 και Πέρα:

Flat Heal (Σταθερή Θεραπεία) ανά Επίπεδο: +1

Max Health Increase (Αύξηση Μέγιστης Υγείας) ανά Επίπεδο: +1



HP παίχτη



Αmmo/Max Ammo παίχτη

Επιβίωση απέναντι στα zombies

3.3 Σύστημα Εμπειρίας (XP)



XP παίχτη

Πολλαπλασιαστής XP ανά Εύρος Επιπέδων:

Levels 1-5: 4x XP

Levels 6-11: 3x XP

Levels 12-20: 2x XP

Levels 21-40: 1.5x XP

Levels 41+: Κανονικό XP

Standard XP gain (Προκαθορισμένες Αξίες XP) ανά Σκοτωμένο Ζόμπι (Επίπεδο 41+):

Old Crone: 4 XP

Nerd: 4 XP

GreyBeard: 4 XP

Butcher: 10 XP

Brute: 10 XP

Torch (Special Zombie): 4 XP

3.4. Τύποι και Ιδιότητες Ζόμπι

Old Crone



Speed: 2.5

Damage: 10

HP: 60

Nerd



Speed: 4

Damage: 10

HP: 40

Πτυχιακή Διατριβή
GreyBeard

Σκαρλής Θρασύβουλος



Speed: 4
Damage: 10
HP: 50

Butcher



Speed: 2
Damage: 20
HP: 150

Επιβίωση απέναντι στα zombies

Πτυχιακή Διατριβή **Brute**

Σκαρλής Θρασύβουλος



Speed: 2
Damage: 20
HP: 120

Torch (Ειδικό Ζόμπι):



Speed: 5.5
Damage: 5
HP: 30

Ειδικές Ικανότητες:

Ρίχνει ένα Health Pack όταν πεθαίνει, θεραπεύοντας τον παίκτη για 50 Flat HP.

Θεραπεύει τον παίκτη για 10% Max Health(Μέγιστης Υγείας) όταν πεθαίνει.

Είναι ευδιάκριτο καθώς το δαδί που κρατάει είναι αναμμένο, ώστε να εντοπίζεται ευκολότερα από τον παίκτη.

3.5. Συμβάν "Loud Scream": Μηχανισμός Ενίσχυσης Ζόμπι

Event Activation (Ενεργοποίηση Συμβάντος): Το "Loud scream" ακούγεται σε συγκεκριμένα διαστήματα (τύπος= καθυστέρηση 60 δευτερόλεπτα στην αρχή, και μετά κάθε 3 λεπτά. Κάθε λεπτό υπάρχει 50% πιθανότητα να ακουστεί).

Effect Duration (Διάρκεια Επιδράσεων): 50 δευτερόλεπτα

Επιδράσεις Ενίσχυσης στα Ζόμπι:

- Doubling the Speed (Διπλασιασμός Ταχύτητας Κίνησης)
- +15 Flat Damage (Πρόσθετη Ζημιά: +15 στη βασική τους ζημιά)

```

10 [SerializeField] private float chanceToPlay = 0.5f; // Chance to play sound randomly within the interval (0 to 1)
11 [SerializeField] private float buffDuration = 50f; // Duration for the zombie buff in seconds
12
13 @ Unity Message | OReferences
14 void Start()
15 {
16     audioSource = GetComponent();
17
18     if (audioSource == null)
19     {
20         Debug.LogError("AudioSource component is missing on this GameObject.");
21     }
22
23     StartCoroutine(SoundPlaybackRoutine());
24 }
25
26 I reference
27 IEnumerator SoundPlaybackRoutine()
28 {
29     // Wait for the initial delay before starting any playback
30     yield return new WaitForSeconds(initialDelay);
31
32     // Start the regular interval timer
33     while (true)
34     {
35         // Play sound and apply buffs
36         PlaySound();
37         ApplyBuffToZombies();
38         yield return new WaitForSeconds(regularInterval);
39
40         // Randomly decide whether to play the sound within the randomInterval timeframe
41         float randomTime = Random.Range(0f, randomInterval);
42         yield return new WaitForSeconds(randomTime);
43
44         // Random chance to play the sound within this interval
45         if (Random.value <= chanceToPlay)
46         {
47             PlaySound();
48             ApplyBuffToZombies();
49         }
50     }
51 }

```

4 Μαθηματικά του παιχνιδιού.

Όσον αφορά τα απολύτως βασικά, το παιχνίδι στον πυρήνα του είναι ιδιαίτερα απλό.

Ο παίχτης έχει συγκεκριμένη υγεία(HP) και η κάθε σφαίρα κάνει 15 damage(ζημιά). Το ίδιο ισχύει και για τα ζόμπι. Άρα όταν η σφαίρα έρχεται σε επαφή με το zombie και αντίστοιχα μια επίθεση από το ζόμπι έρθει σε επαφή με τον παίχτη, μια αφαίρεση είναι αρκετή για να υπολογίσουμε το υπολειπόμενο HP(υγεία) του zombie και του παίχτη αντίστοιχα. Αυτό όμως που αξίζει να αναλύσουμε εδώ είναι το spawn rate του κάθε zombie. Δηλαδή το πόσο γρήγορα έρχεται στον κόσμο ένα νέο zombie, αριθμός που είναι μεταβλητός με την πάροδο του χρόνου. Συγκεκριμένα:

4.1 Σύστημα Εμφάνισης Ζόμπι

Το παιχνίδι διαθέτει ένα δυναμικό σύστημα εμφάνισης, με κάθε τύπο ζόμπι να έχει ένα μοναδικό βασικό διάστημα εμφάνισης (Spawn Rate). Καθώς το παιχνίδι προχωρά, τα διαστήματα εμφάνισης σταδιακά μειώνονται, αυξάνοντας τη συνολική δυσκολία και την ένταση του horde mode. Ωστόσο, για να αποτραπεί η υπερβολικά γρήγορη εμφάνιση, κάθε διάστημα εμφάνισης έχει ένα ελάχιστο όριο 0,1 δευτερολέπτων.

Τύπος Υπολογισμού Χρόνου Εμφάνισης

Για κάθε ζόμπι, το διάστημα εμφάνισης υπολογίζεται ως εξής:

Χρόνος Εμφάνισης=μέγιστο(Βασικός Χρόνος Εμφάνισης–(Ρυθμός Μείωσης Χρόνος Παιχνιδιού'),0.1δευτερόλεπτα) ×

Όπου:

- *Βασικός Χρόνος Εμφάνισης* είναι το αρχικό διάστημα μεταξύ εμφανίσεων ζόμπι (σε δευτερόλεπτα).
- *Ρυθμός Μείωσης* είναι ο ρυθμός με τον οποίο μειώνεται το διάστημα εμφάνισης με την πάροδο του χρόνου.
- *Χρόνος Παιχνιδιού* είναι ο συνολικός χρόνος από την αρχή του παιχνιδιού (σε δευτερόλεπτα).

Τύπος Ζόμπι	Βασικός Χρόνος Εμφάνισης	Ρυθμός Μείωσης	Ελάχιστος Χρόνος Εμφάνισης
Torch	100 δευτερόλεπτα	0.001	0.1 δευτερόλεπτα
Butcher	50 δευτερόλεπτα	0.001	0.1 δευτερόλεπτα
Nerd	10 δευτερόλεπτα	0.01	0.1 δευτερόλεπτα
Brute	20 δευτερόλεπτα	0.01	0.1 δευτερόλεπτα
Greybeard	10 δευτερόλεπτα	0.01	0.1 δευτερόλεπτα
Old Crone	20 δευτερόλεπτα	0.01	0.1 δευτερόλεπτα

Κατανομή Σημείων Εμφάνισης:

- Σημείο Εμφάνισης *Torch*: 1 σημείο εμφάνισης
- Σημεία Εμφάνισης *Nerd*: 2 σημεία εμφάνισης
- Σημείο Εμφάνισης *Butcher*: 1 σημείο εμφάνισης
- Σημείο Εμφάνισης *Old Crone*: 1 σημείο εμφάνισης
- Σημεία Εμφάνισης *Greybeard*: 2 σημεία εμφάνισης
- Σημεία Εμφάνισης *Brute*: 4 σημεία εμφάνισης

Συμπεριφορά Κίνησης μετά την Εμφάνιση:

Μετά την εμφάνιση ενός ζόμπι, το σημείο εμφάνισής του μετακινείται ώστε να αποφεύγεται η επικάλυψη των εμφανίσεων και να διασφαλίζεται η εξάπλωση των ζόμπι στον χάρτη. Το μοτίβο κίνησης είναι το εξής:

1. Βήμα 1: Μετακίνηση δεξιά από την αρχική θέση κατά έναν προκαθορισμένο αριθμό μονάδων.
2. Βήμα 2: Επιστροφή στην αρχική θέση με μετακίνηση αριστερά.
3. Βήμα 3: Μετακίνηση προς τα εμπρός από την αρχική θέση.
4. Βήμα 4: Επιστροφή στην αρχική θέση με μετακίνηση προς τα πίσω.

Τα ζόμπι στη συνέχεια χρησιμοποιούν αόρατες ράμπες για να μπουν στην περιοχή του χάρτη που είναι προσβάσιμη στον παίκτη.

```

54 void TeleportSpawnPoint()
55 {
56     public /*Calculate the new position based on the current teleport direction*/
57     Vector3 newPosition = originalPosition;
58     switch (teleportDirection)
59     {
60         case 0: // Right
61             newPosition += Vector3.right * teleportDistance;
62             break;
63         case 1: // Left
64             newPosition -= Vector3.right * teleportDistance;
65             break;
66         case 2: // Down
67             newPosition -= Vector3.forward * teleportDistance;
68             break;
69         case 3: // Up
70             newPosition += Vector3.forward * teleportDistance;
71             break;
72     }
73
74     // Teleport the spawn point to the new position
75     transform.position = newPosition;
76
77     // Update the teleport direction for the next teleport
78     teleportDirection = (teleportDirection + 1) % 4;
79 }
80
81

```

Παράδειγμα Υπολογισμού Εμφάνισης Ζόμπι Torch

Για το Torch Zombie, ο βασικός χρόνος εμφάνισης είναι 100 δευτερόλεπτα, και ο ρυθμός μείωσης είναι 0.001. Μετά από 200 δευτερόλεπτα παιχνιδιού, ο χρόνος εμφάνισης θα υπολογιστεί ως εξής:
 Χρόνος Εμφάνισης = $\max(100 - (0.001 \times 200), 0.1) = \max(99.8, 0.1) = 99.8$ δευτερόλεπτα
 Χρόνος Εμφάνισης = $\max(100 - (0.001 \times 200), 0.1) = \max(99.8, 0.1) = 99.8$ δευτερόλεπτα

Αυτή η τιμή θα συνεχίσει να μειώνεται με την πάροδο του χρόνου, αλλά δεν θα πέσει ποτέ κάτω από τον ελάχιστο χρόνο εμφάνισης των 0.1 δευτερολέπτων.

Με την εισαγωγή αυτού του δυναμικού συστήματος εμφάνισης, το παιχνίδι διατηρεί μια σταθερή αύξηση της δυσκολίας με την πάροδο του χρόνου, αναγκάζοντας τους παίκτες να προσαρμόζονται και να διαχειρίζονται τις αυξανόμενες και πιο ποικιλόμορφες κυματομορφές των ζόμπι.

```

28 void Update()
29 {
30     // Update the time since the last spawn
31     timeSinceLastSpawn += Time.deltaTime;
32
33     // Check if it's time to spawn a new enemy
34     if (timeSinceLastSpawn >= currentSpawnRate)
35     {
36         SpawnEnemy();
37         timeSinceLastSpawn = 0f;
38     }
39
40     // Increase spawn rate over time
41     currentSpawnRate -= spawnRateIncreasePerSecond * Time.deltaTime;
42     currentSpawnRate = Mathf.Max(currentSpawnRate, 0.1f); // Ensure spawn rate doesn't go below 0.1 second
43 }
44

```

5 Scenes

5.1 Οι σκηνές/Scenes στο Unity

Τα Scenes στο Unity αποτελούν ουσιαστικά δομικές μονάδες ενός παιχνιδιού ή εφαρμογής, και κάθε σκηνή μπορεί να θεωρηθεί σαν ένα ξεχωριστό επίπεδο, μενού ή περιβάλλον. Αποτελούν έναν τρόπο ομαδοποίησης αντικειμένων (GameObjects), στοιχείων φωτισμού, κάμερας, και άλλων ρυθμίσεων σε μία ενιαία συλλογή.

Πώς χρησιμοποιούνται τα Scenes στο Unity:

- **Δημιουργία Επίπεδων/Σταδίων:** Σε ένα παιχνίδι, κάθε επίπεδο συνήθως αντιπροσωπεύεται από μία σκηνή. Κάθε σκηνή μπορεί να περιέχει διαφορετικά στοιχεία όπως τοπογραφία, εχθρούς, στόχους και περιβάλλον. Αυτό βοηθά στη διαχείριση του περιεχομένου του παιχνιδιού, καθώς κάθε σκηνή περιλαμβάνει μόνο τα απαραίτητα στοιχεία για το εκάστοτε στάδιο.
- **Μενού και Διεπαφές Χρήστη (UI):** Συχνά, δημιουργούνται ξεχωριστές σκηνές για μενού, όπως η αρχική οθόνη, το μενού επιλογών ή το μενού παύσης. Έτσι, διαχωρίζεται ο κώδικας και τα αντικείμενα του UI από τον βασικό κόσμο του παιχνιδιού.
- **Διαχείριση Φόρτωσης Πόρων:** Οι σκηνές επιτρέπουν να φορτώνονται μόνο οι απαραίτητοι πόροι και αντικείμενα κάθε φορά, εξοικονομώντας μνήμη και βελτιώνοντας την απόδοση. Αντί να υπάρχει όλο το περιεχόμενο του παιχνιδιού συνεχώς στη μνήμη, μόνο η ενεργή σκηνή (ή οι σκηνές που χρειάζονται) φορτώνεται και προβάλλεται.

Εναλλαγή Μεταξύ Scenes

Η εναλλαγή μεταξύ σκηνών γίνεται μέσω του Scene Management API του Unity, το οποίο προσφέρει μεθόδους για τη φόρτωση, την εναλλαγή και τη διαχείριση των σκηνών:

- **SceneManager.LoadScene:** Η βασική μέθοδος για εναλλαγή σκηνών είναι η `LoadScene()`, η οποία φορτώνει μία νέα σκηνή και αντικαθιστά την προηγούμενη. Αυτή η μέθοδος είναι χρήσιμη όταν θέλουμε μια πλήρη εναλλαγή, όπως από ένα επίπεδο σε άλλο ή από το μενού στην κύρια σκηνή παιχνιδιού.
- **SceneManager.LoadSceneAsync:** Για περιπτώσεις όπου η σκηνή είναι πολύ "βαριά" και χρειάζεται χρόνος για να φορτωθεί, χρησιμοποιείται η ασύγχρονη φόρτωση (`LoadSceneAsync()`), η οποία φορτώνει τη σκηνή στο παρασκήνιο και μπορεί να προσθέσει μία μπάρα προόδου για να ενημερώνεται ο παίκτης. Ιδανική για παιχνίδια με μεγάλους κόσμους ή πολλά γραφικά.
- **Additive Mode:** Η `LoadSceneMode.Additive` επιτρέπει την φόρτωση μιας σκηνής παράλληλα με την τρέχουσα σκηνή, ώστε να μπορούν να συνυπάρχουν πολλαπλές σκηνές στην ίδια χρονική στιγμή. Αυτό είναι χρήσιμο για τη φόρτωση επιπλέον τμημάτων, όπως νέα στοιχεία στον ίδιο χάρτη ή για τη δημιουργία επιπέδων που φορτώνονται δυναμικά.

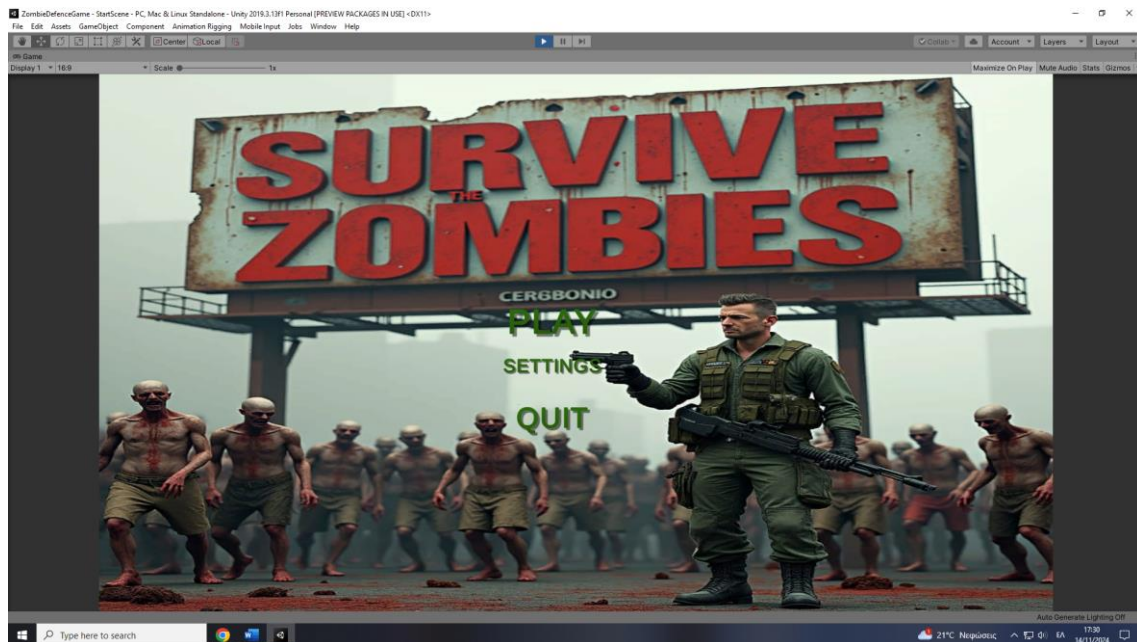
5.2 Τα Scenes του παιχνιδιού

Στο παιχνίδι, βρίσκονται 3 διαφορετικά scenes. Το πρώτο (`StartScene`) είναι το αρχικό μενού του αρχικού μενού του παιχνιδιού και έχει τις επιλογές `PLAY`, `SETTINGS` και `QUIT`.

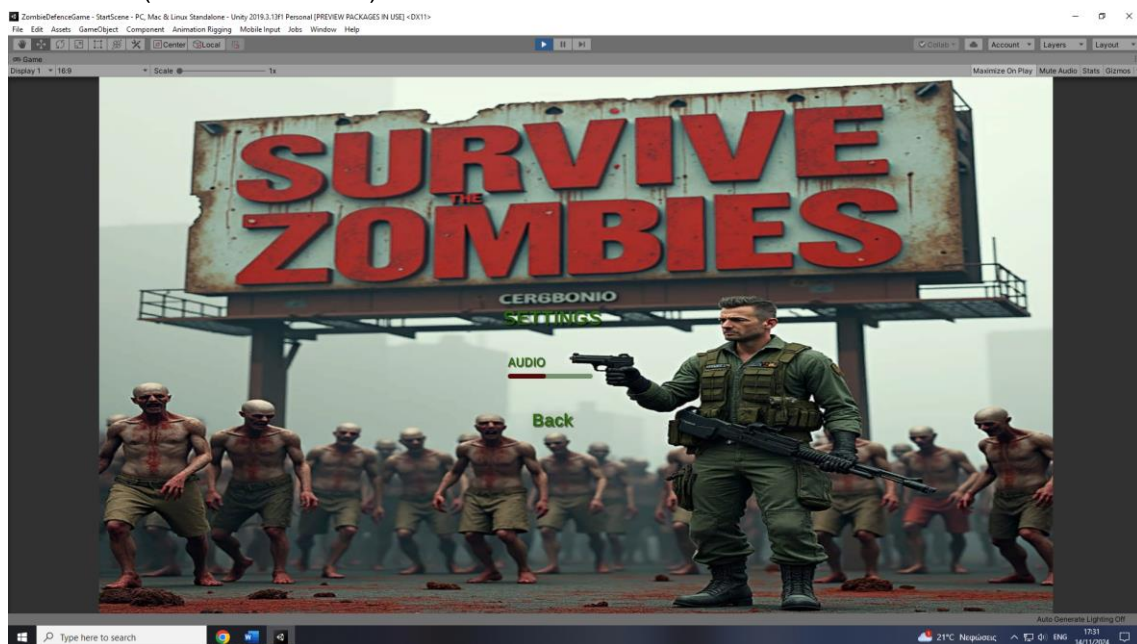
Η επιλογή `QUIT` θα κλείσει εντελώς την εφαρμογή, ενώ η επιλογή `SETTINGS` μεταφέρει τον χρήστη σε ένα δεύτερο μενού που μπορεί να πειράξει την ένταση του ήχου από ανάλογο scrollbar. Τέλος το `Play` είναι υπεύθυνο για τη μεταφορά στο κύριο scene και ουσιαστικά την έναρξη του παιχνιδιού.

Πτυχιακή Διατριβή StartScene (αρχικό Μενού)

Σκαρλής Θρασύβουλος



StartScene (SETTINGS Μενού)



Το MainScene είναι ουσιαστικά ολόκληρο το παιχνίδι με ότι αυτό περιέχει.(όπως έχει αναλυθεί λεπτομερώς στο κεφάλαιο 3).

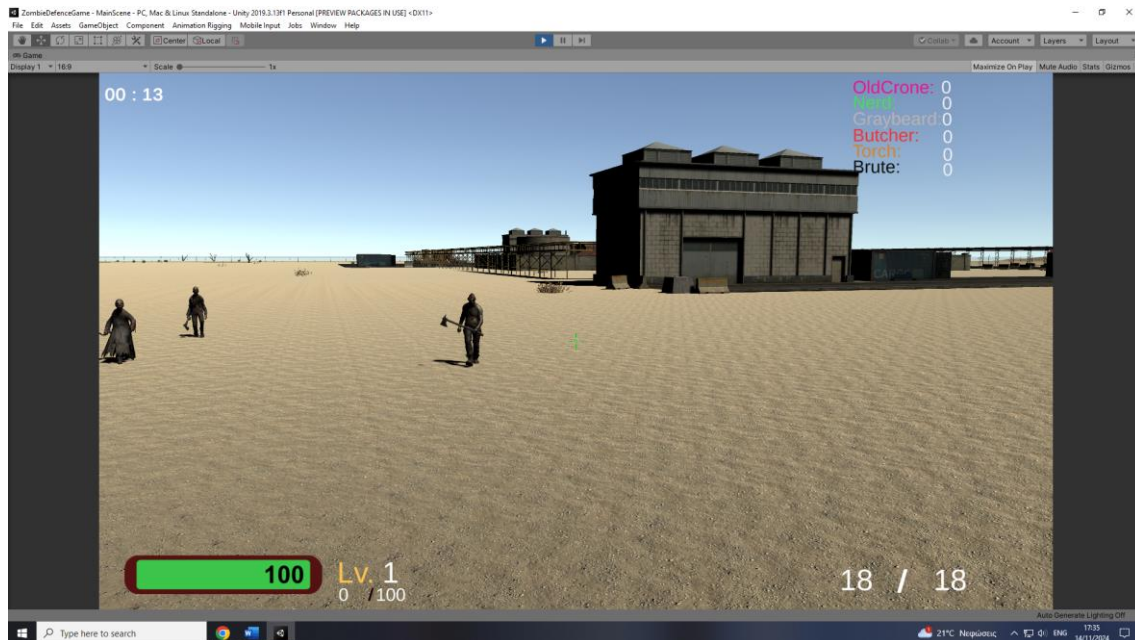
Πτυχιακή Διατριβή MainScene

Σκαρλής Θρασύβουλος



Εννοείται στο Main scene έχει συμπεριληφθεί και ένα Pause menu όπου ο παίχτης πατώντας το πλήκτρο ESC μπορεί να σταματήσει το παιχνίδι , φυσικά να αλλάξει και πάλι την ένταση του ήχου ή ακόμα και να το κλείσει πατώντας το κουμπί Quit.

MainScene(Pause Μενού)



Το τρίτο και τελευταίο scene του παιχνιδιού, είναι το DefeatScene, το οποίο βλέπει ο χρήστης όταν χάσει στο παιχνίδι, δηλαδή όταν το HP του πέσει κάτω από το μηδέν. Το scene αυτό περιλαμβάνει το πλήκτρα Play Again, το οποίο επιτρέπει στον χρήστη να ξαναπαίξει το παιχνίδι(φορτώνοντας) το MainScene και το κουμπί Quit που δίνει τη δυνατότητα στον χρήστη να κλείσει τελείως το παιχνίδι. Σε αυτό το scene έχει συμπεριληφθεί και ένα στιγμιότυπο από το

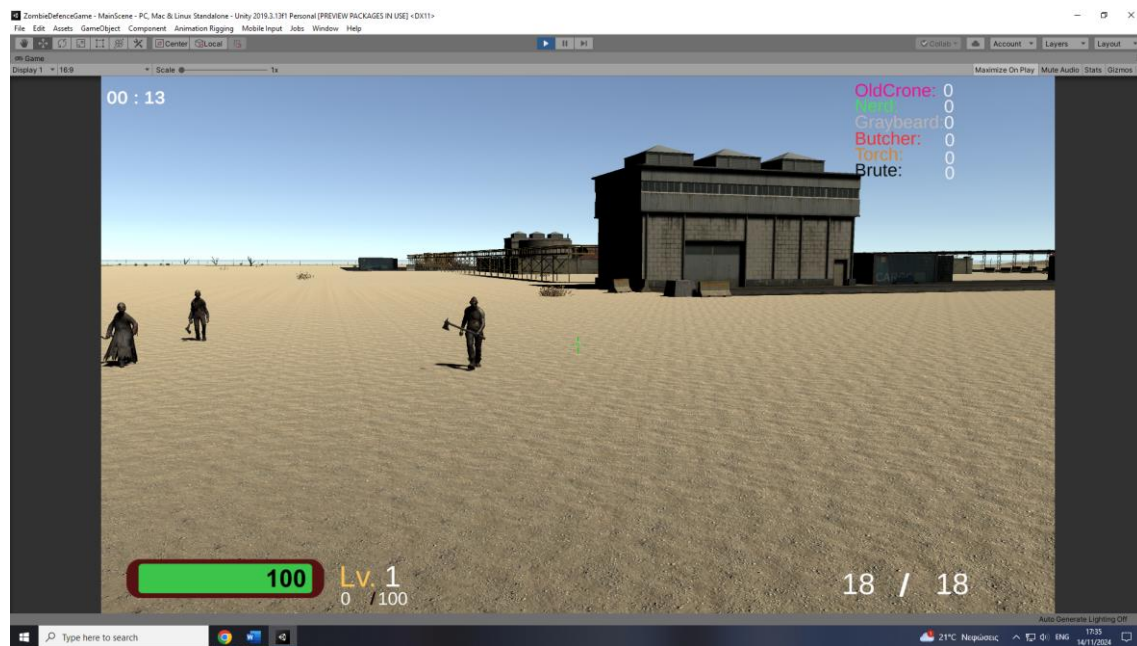
Επιβίωση απέναντι στα zombies

Πτυχιακή Διατριβή

Σκαρλής Θρασύβουλος

τελευταίο δευτερόλεπτο προτού ο χρήστης χάσει, όπου μπορεί να διαβάσει το χρόνο που άντεξε, καθώς επίσης και πόσα αλλά και ποια ζόμπι σκότωσε.

DefeatScene



DefeatScene



Επιβίωση απέναντι στα zombies

6 Animations – Animator

Το Animator στο Unity είναι ένα εργαλείο που χρησιμοποιείται για τη δημιουργία, τη διαχείριση και την αναπαραγωγή κινούμενων σχεδίων (animations) στα αντικείμενα ενός παιχνιδιού. Είναι μέρος του συστήματος Animation του Unity και συνδυάζει τα animations με μηχανισμούς που βασίζονται σε καταστάσεις (state machines).

6.1 Βασικά Στοιχεία του Animator

1. **Animator Component:** Το Animator Component είναι το στοιχείο που προστίθεται σε ένα GameObject για να διαχειρίζεται τα animations του. Συνδέεται με ένα Animator Controller, το οποίο περιέχει όλα τα animations και τους κανόνες εναλλαγής τους.
2. **Animator Controller:** Το Animator Controller είναι ένα αρχείο στο Unity που καθορίζει τις μεταβάσεις μεταξύ των animations και τις συνθήκες υπό τις οποίες θα γίνονται αυτές οι μεταβάσεις. Δημιουργείται μέσω του Animator Window.
3. **Animator Window:** Είναι το περιβάλλον στο Unity όπου σχεδιάζονται τα animations και οι μεταβάσεις τους. Περιλαμβάνει έναν πίνακα που δείχνει τα διάφορα animations states και τις συνδέσεις τους.

6.2 Πως λειτουργεί το Animator

1. **Animation States:** Κάθε animation state αντιπροσωπεύει μια κίνηση ή κατάσταση. Για παράδειγμα, ένα character μπορεί να έχει τα εξής states: Idle, Walking, Running, και Jumping. Το Animator Controller καθορίζει πότε και πώς ο χαρακτήρας θα μεταβεί από το ένα state στο άλλο.
2. **Transitions (Μεταβάσεις):** Οι μεταβάσεις καθορίζουν πώς και πότε ένα αντικείμενο αλλάζει από ένα state σε άλλο. Συνήθως βασίζονται σε παραμέτρους, όπως:
 - **Triggers** Π.χ., ενεργοποίηση ενός "Jump" trigger για μετάβαση σε animation άλματος.
 - **Booleans** Π.χ., isRunning για να καθοριστεί αν το Run animation πρέπει να αναπαραχθεί.
 - **Floats/Integers** Π.χ., μια παράμετρος "Speed" για μετάβαση από Idle σε Walking και Running.
3. **Blending (Συνδυασμός):** Το Animator υποστηρίζει blending μεταξύ animations, δηλαδή ομαλή μετάβαση από το ένα animation στο άλλο. Χρησιμοποιείται για πιο φυσικές κινήσεις, π.χ., από Walking σε Running.

6.3 Χαρακτηριστικά του Animator

1. **Root Motion:** Επιτρέπει στα animations να ελέγχουν την κίνηση ενός αντικείμενου στον χώρο. Χρήσιμο για χαρακτήρες που κινούνται με βάση τα animations τους, π.χ., ένας χαρακτήρας που περπατά.
2. **Avatar Masks:** Χρησιμοποιούνται για την εφαρμογή animations σε συγκεκριμένα μέρη ενός GameObject. Π.χ., ένα animation που κινεί μόνο τα χέρια ενός χαρακτήρα ενώ τα πόδια του παραμένουν σταθερά.

3. **Layers:** Το Animator μπορεί να έχει πολλαπλά στρώματα animations. Π.χ., το βασικό animation layer μπορεί να χειρίζεται τα πόδια, ενώ ένα δεύτερο layer να ελέγχει τα χέρια.
4. **Animation Events:** Τα Animation Events επιτρέπουν την ενεργοποίηση συγκεκριμένων κώδικα ή λειτουργιών σε συγκεκριμένα σημεία του animation. Π.χ., να παίζεται ένας ήχος όταν ο χαρακτήρας πατάει στο έδαφος.

6.4 Animation Rigging

Το rigging είναι η διαδικασία δημιουργίας ενός "σκελετού" (rig) για έναν χαρακτήρα ή αντικείμενο, το οποίο χρησιμοποιείται για την κινούμενη προσαρμογή του. Περιλαμβάνει bones (οστά), joints (αρθρώσεις) και constraints (περιορισμούς) που καθορίζουν πώς κινούνται και επηρεάζουν το αντικείμενο.

Το Animation Rigging Package του Unity προσθέτει μια σειρά εργαλείων για τη διαχείριση constraints και τη δημιουργία δυναμικών rigs. Περιλαμβάνει έτοιμα components για τη δημιουργία constraints όπως :

- **Two Bone IK constraint:** Ρυθμίζει την κίνηση ενός χεριού ή ποδιού για να στοχεύει σε ένα σημείο.
- **Multi-Parent Constraint:** Συνδέει ένα bone με πολλαπλά targets.
- **Twist Chain Constraint:** Εφαρμόζει ομοιόμορφη περιστροφή σε μια αλυσίδα οστών.
- **Damped Transform:** Δημιουργεί πιο φυσικές κινήσεις με ελαστικότητα.

Τα Constraints είναι περιορισμοί που προστίθενται στα bones για να καθορίσουν τη συμπεριφορά τους. Π.χ., ένα χέρι μπορεί να περιοριστεί ώστε να παραμένει σε μια συγκεκριμένη θέση ή κατεύθυνση ανεξάρτητα από την υπόλοιπη κίνηση.

Με το Animation Rigging, μπορούν να προσαρμοστούν animations κατά τη διάρκεια του παιχνιδιού, επιτρέποντας στους χαρακτήρες να αντιδρούν δυναμικά στο περιβάλλον (π.χ., να πιάνουν αντικείμενα ή να κοιτάζουν ένα στόχο), κάτι που ονομάζεται dynamic adjustment.

Βασικά Components στο Animation Rigging:

1. **Rig Builder:** Το Rig Builder προστίθεται στο GameObject του χαρακτήρα και επιτρέπει τη σύνδεση και διαχείριση πολλαπλών rigs.
2. **Rig Layers:** Οι στρώσεις (layers) στο rig επιτρέπουν διαφορετικές προτεραιότητες και blending ανάμεσα σε constraints.
3. **Target Object:** Χρησιμοποιούνται αντικείμενα ως targets για constraints, π.χ., ένα χέρι να ακολουθεί τη θέση ενός κύβου.
4. **Playable API:** Επιτρέπει προγραμματιστικό έλεγχο και blending constraints μέσω κώδικα, για δυναμικές αλληλεπιδράσεις.

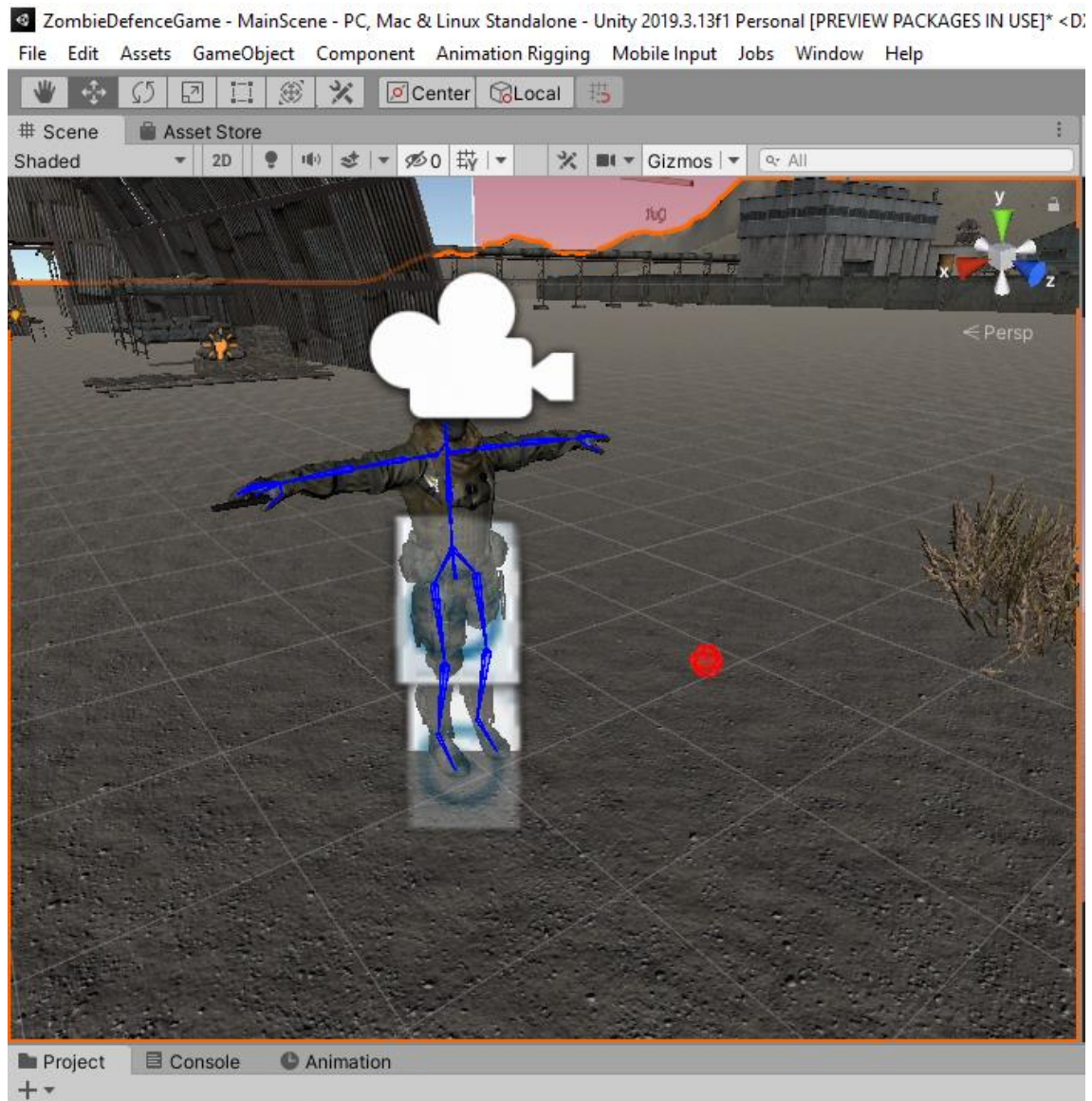
6.5 Πλεονεκτήματα του Animation Rigging

Το Animation Rigging είναι ένα ισχυρό εργαλείο που εμπλουτίζει τις δυνατότητες animation στο Unity, επιτρέποντας στους δημιουργούς να συνδυάζουν ρεαλισμό και δραστηριότητα στις κινήσεις των χαρακτήρων και αντικειμένων τους. Βασικά πλεονεκτήματα είναι τα ακόλουθα:

1. **Δυναμικότητα:** Επιτρέπει την προσαρμογή κινήσεων σε πραγματικό χρόνο, κάνοντας τους χαρακτήρες να φαίνονται πιο φυσικοί και ρεαλιστικοί.
2. **Αυξημένος έλεγχος:** Συνδυάζει προκαθορισμένα Animations με constraints για πιο πλούσιες αλληλεπιδράσεις.
3. **Ευελιξία:** Χρησιμοποιείται για οποιοδήποτε αντικείμενο χρειάζεται σύνθετη κίνηση, από χαρακτήρες μέχρι μηχανές.

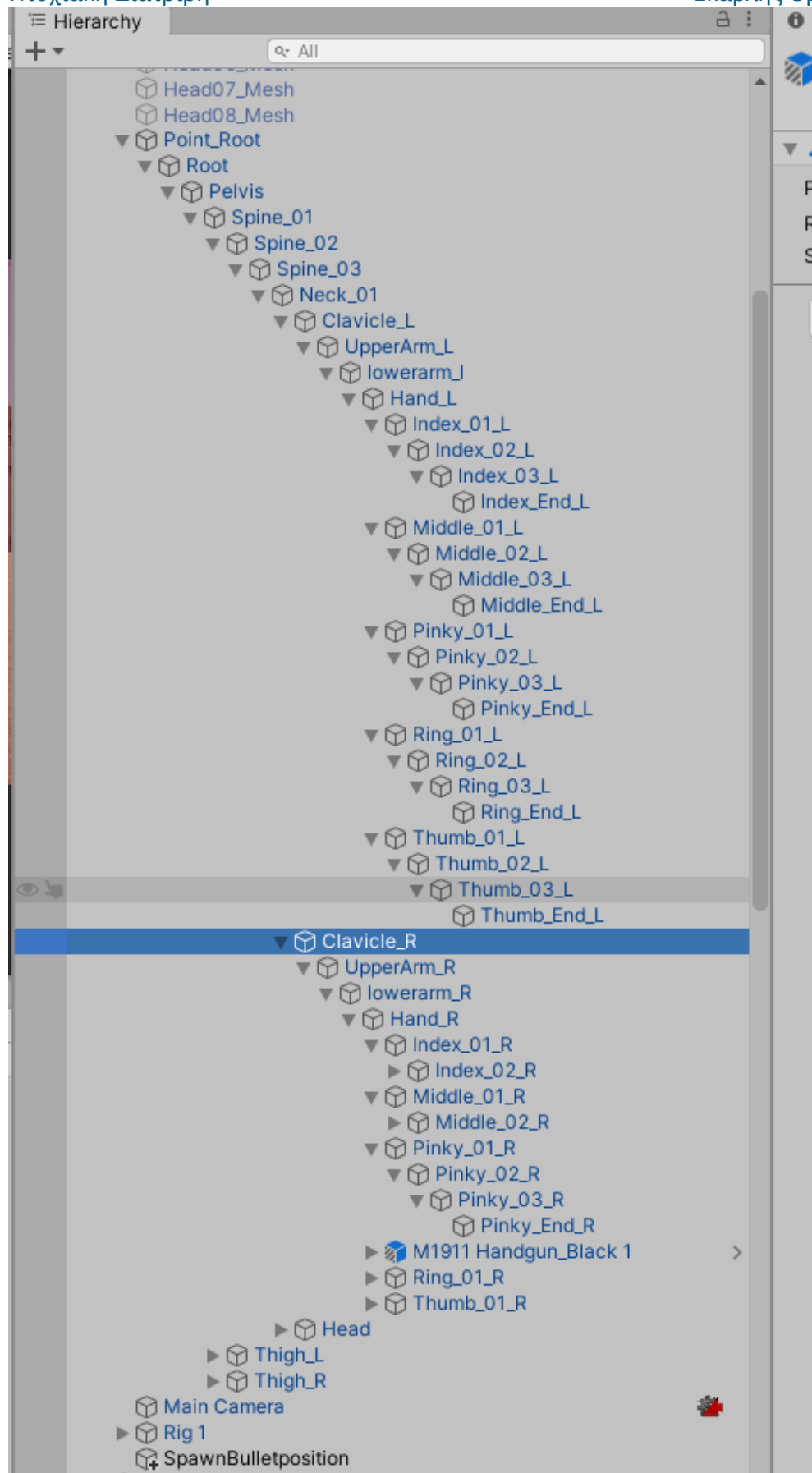
6.6 Animator και Animation Rigging στο “Survive The Zombies”

Στο παιχνίδι χρησιμοποιήθηκε το animation rigging μόνο στο asset του παίχτη καθώς όλα τα animations των ζόμπι ήταν άρτια εξαρχής και δεν χρειάστηκε κάτι παραπάνω όσων αφορά το rigging. Στην παρακάτω εικόνα ο μπλε σκελετός μέσα στο μοντέλο είναι ακριβώς αυτό, δηλαδή ένα rig builder με τόσο multiparent-constraint, όσο και Two Bone IK constrain. Το πρώτο χρησιμοποιήθηκε για ομαδοποίηση μικρότερων οστών(πχ δάχτυλα), ενώ το δεύτερο κυρίως για τα χέρια και τα πόδια. Η μπλε μπάλα που εμφανίζεται μπροστά από τον παίχτη πρόκειται για το target object και έχει τοποθετηθεί σε αυτό το σημείο, έτσι ώστε το κεφάλι του παίχτη να γυρίζει ανάλογα με του που στοχεύει ο παίχτης.

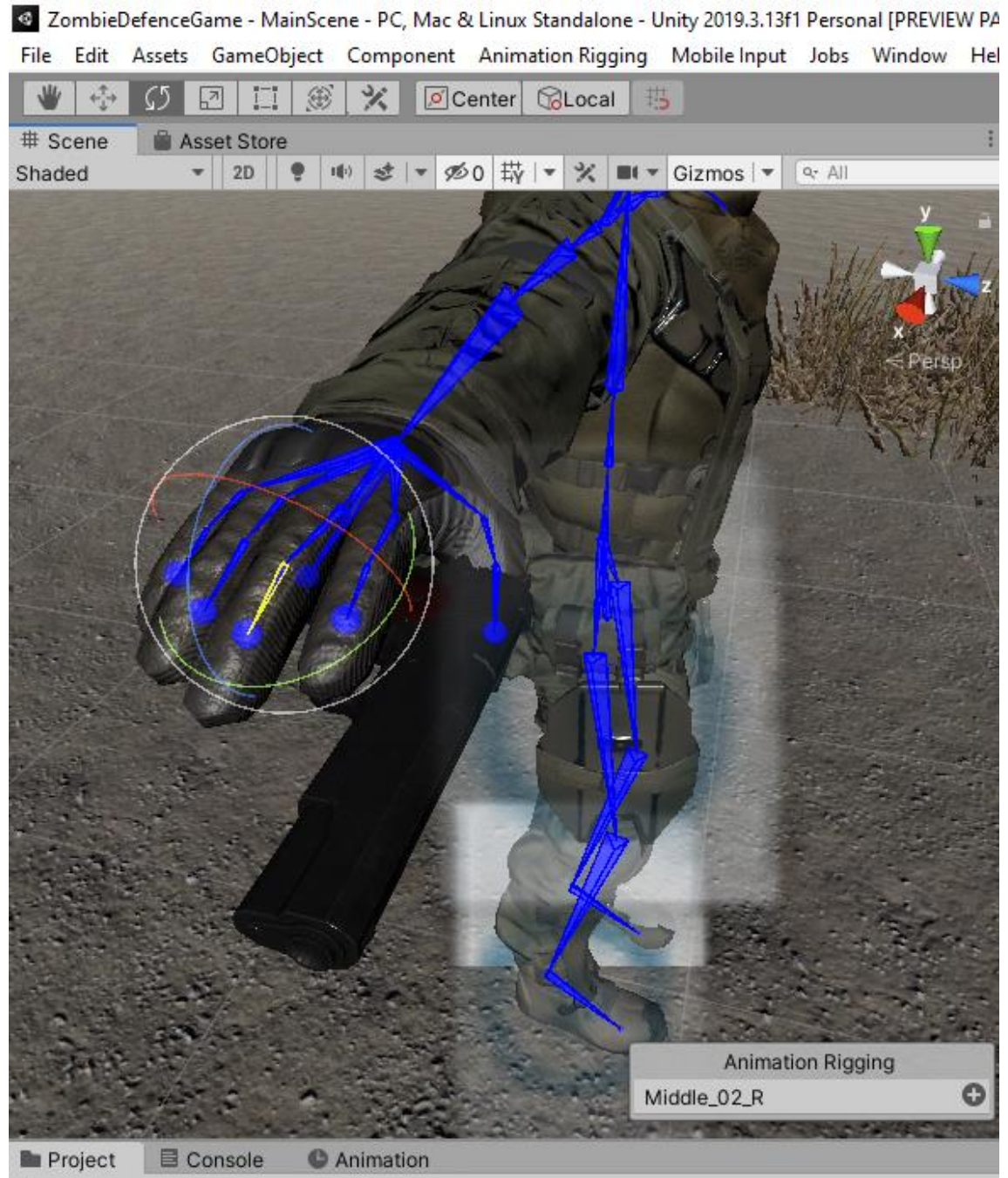


Ενώ στην παρακάτω εικόνα από το inspector του Unity φαίνεται η λεπτομέρεια και η ακρίβεια που μας επιτρέπει αυτό το εργαλείο να δώσουμε στα animations μας. πχ έχουμε πρόσβαση ακόμα και σε κάθε δάχτυλο ξεχωριστά.

Δηλαδή μια πρόσβαση σε κάθε ανθρώπινο οστό, εφόσον και το μοντέλο μας είναι humanoid μοντέλο. Το ίδιο κάνει ξεκάθαρο και η μετέπειτα εικόνα.

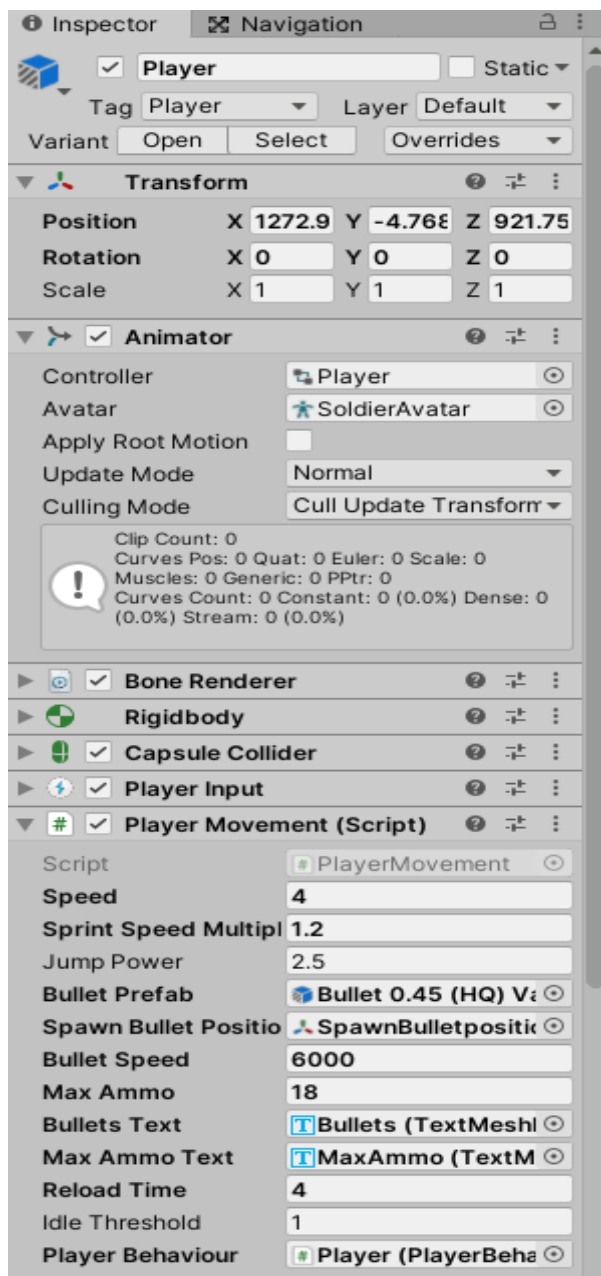


Επιβίωση απέναντι στα zombies



Animator Overview για τον παίκτη

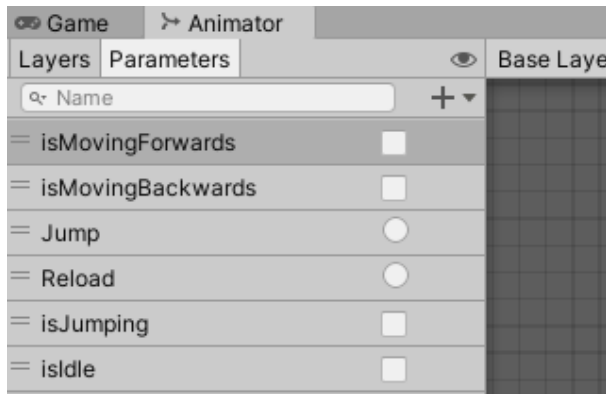
Για τα animations του το παίκτη δεν χρησιμοποιήθηκε το RootMotion όπως φαίνεται και στην παρακάτω εικόνα όπου το ανάλογο κουτάκι δεν έχει επιλεγεί. Αυτό έγινε για το γεγονός ότι πολλά από τα animations, καθώς και όλες οι κινήσεις που κάνει ο παίκτης γίνονται μέσω κώδικα. Τα scripts PlayerMovement και PlayerBehavior είναι υπεύθυνα για όλες τις δράσεις του παίκτη. Τα scripts αυτά, θα αναλυθούν στο επόμενο κεφάλαιο, καθώς και άλλα τα οποία δεν αφορούν τον παίκτη, λεπτομερώς.



Πτυχιακή Διατριβή

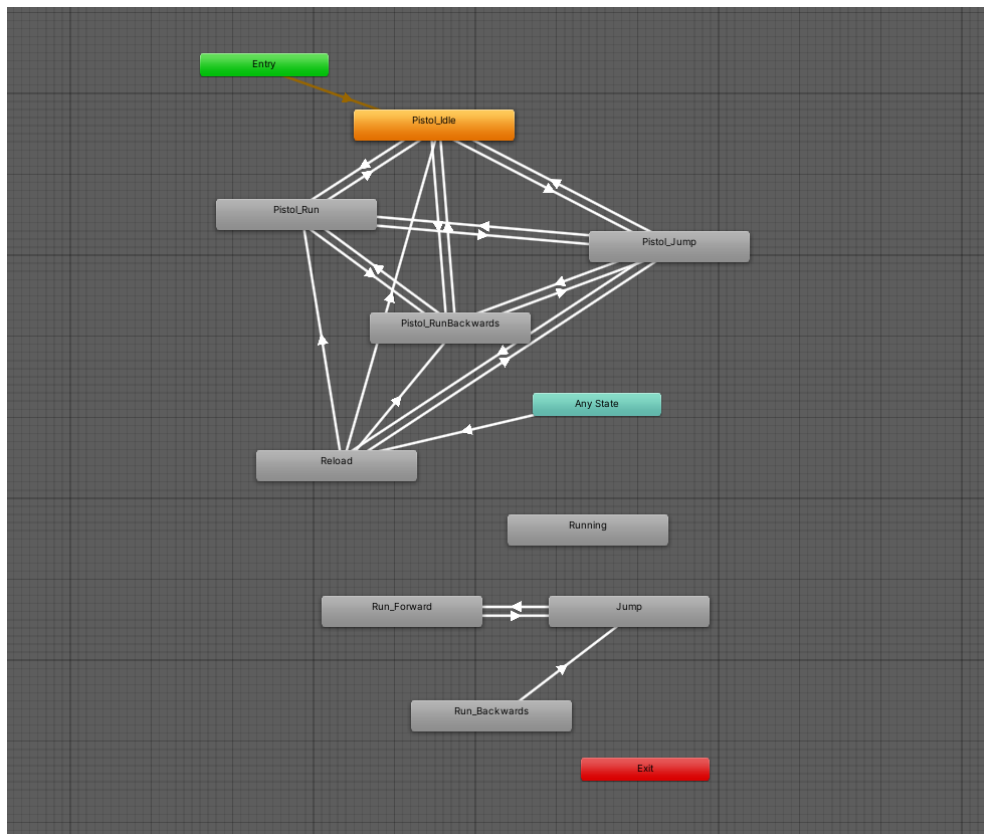
Σκαρλής Θρασύβουλος

Για το αν Animator του παίκτη χρησιμοποιήθηκε μόνο ένα Layer, ώστε να παραμείνει ιδιαίτερα απλό. Χρησιμοποιήθηκαν επίσης, τέσσερα Boolean parameters και δύο triggers. Αυτά είναι με τη σειρά: isMovingForwards για όταν ο παίχτης προχωράει με κατεύθυνση προς τα μπροστά, isMovingBackwards για όταν προχωράει με κατεύθυνση προς τα πίσω, isJumping για όταν πηδάει και isIdle για όταν δεν ανιχνεύει καμία απολύτως κίνηση ή εντολή. Ενώ τα Triggers είναι τα Jump πάλι για όταν πηδάει και Reload για όταν το σπλίζει το γεμιστήρα του όπλου του.



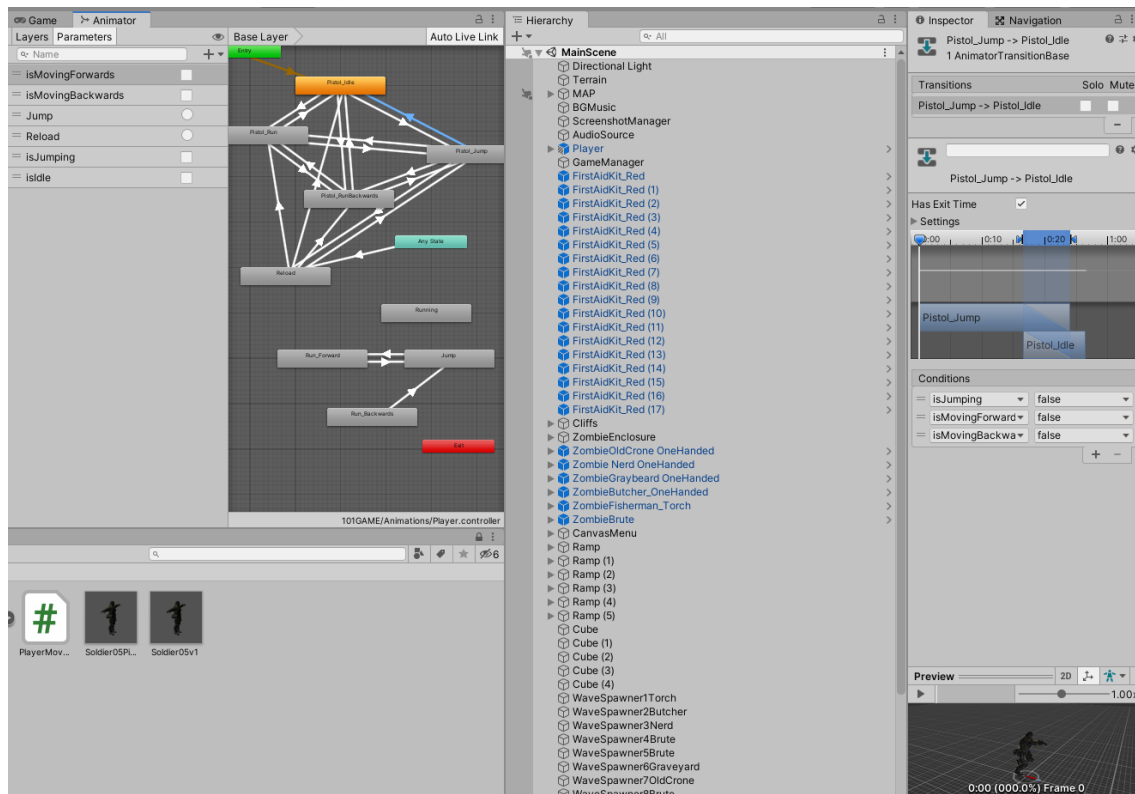
BaseLayer Animator παίχτη

Στην από κάτω εικόνα φαίνονται τα animation states του παίχτη. Τα βελάκια συμβολίζουν όλες τις επιτρεπτές μεταβάσεις. Το Entry state είναι απαραίτητο και από εκεί πάντα ξεκινάνε οι μεταβάσεις, ενώ το πορτοκαλί state είναι το πρώτο animation που θα παιχτεί, με την εκκίνηση του παιχνιδιού ανεξαρτήτως. Το γαλάζιο που ονομάζεται "Any state", σημαίνει ότι σε όποιο animation state και να βρίσκεται ο παίχτης αν εκπληρωθεί το συγκεκριμένο condition θα με μεταβεί εκεί που πηγαίνει το βελάκι. Παρατηρούμε ότι υπάρχει και ένα animation state που δεν έχει κανένα βελάκι, αυτό γιατί είναι ένα animation που δεν χρησιμοποιείται.



Στη συνέχεια θα εξηγήσουμε Ορισμένες από τις μεταβάσεις που κάνει ο παίκτης έτσι ώστε από ένα animation να μεταφερθεί στο επόμενο, με τη σειρά στο επόμενο, πάλι πίσω ίσως, και ούτω καθεξής

Η πρώτη μετάβαση που θα αναλύσουμε είναι αυτή όπου ο παίκτης βρίσκεται σε Jump state και δεν εκτελεί καμία πράξη οπότε πρέπει να γυρίσει στο animation του idle state. Αυτό επιτυγχάνεται με τρία Boolean Parameters όπου όλα είναι false. (isJumping, isMovingForwards, isMovingBackwards), όπως φαίνεται και στην παρακάτω εικόνα.

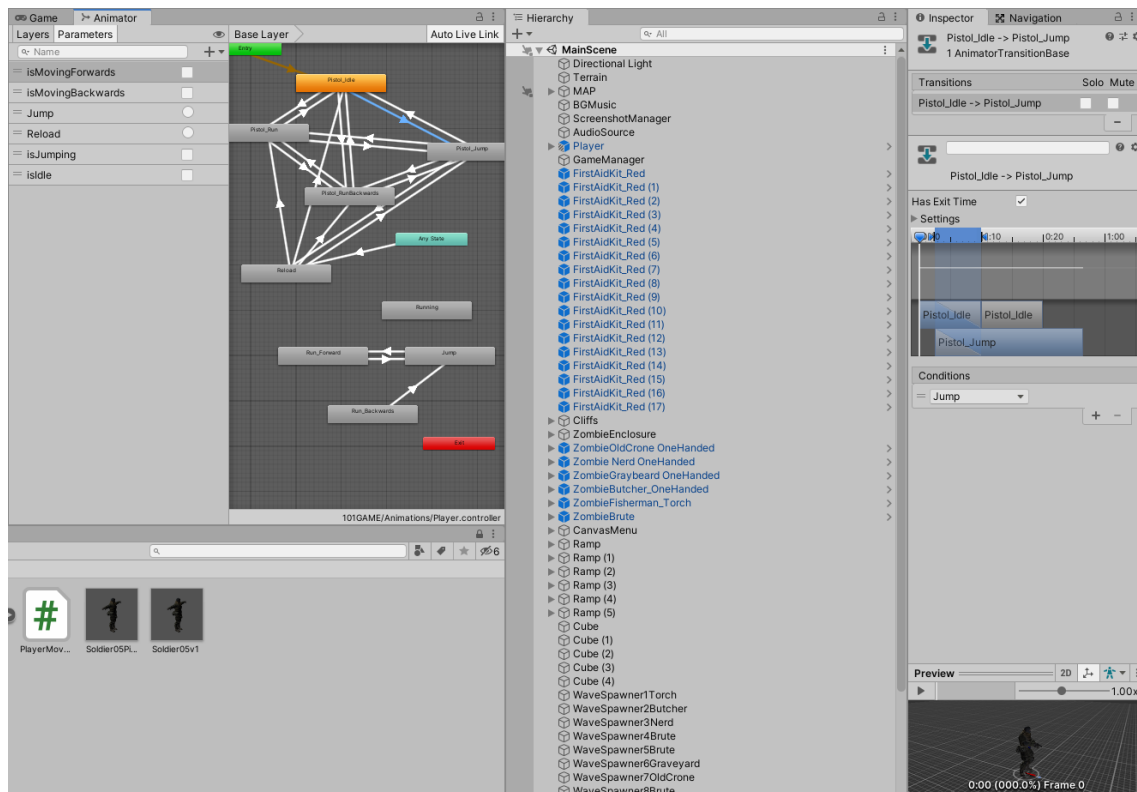


(έμφαση στο βελάκι της μετάβασής(μπλε χρώμα), αλλά και στα conditions στο inspector δεξιά)

Πτυχιακή Διατριβή

Σκαρλής Θρασύβουλος

Η επόμενη μετάβαση που αξίζει να μελετηθεί είναι μια πολύ πιο απλή, και είναι αυτή από το idle state στο Jump state. ένα μόνο trigger parameter είναι αρκετό για αυτή τη μετάβαση και αυτό είναι το Jump.

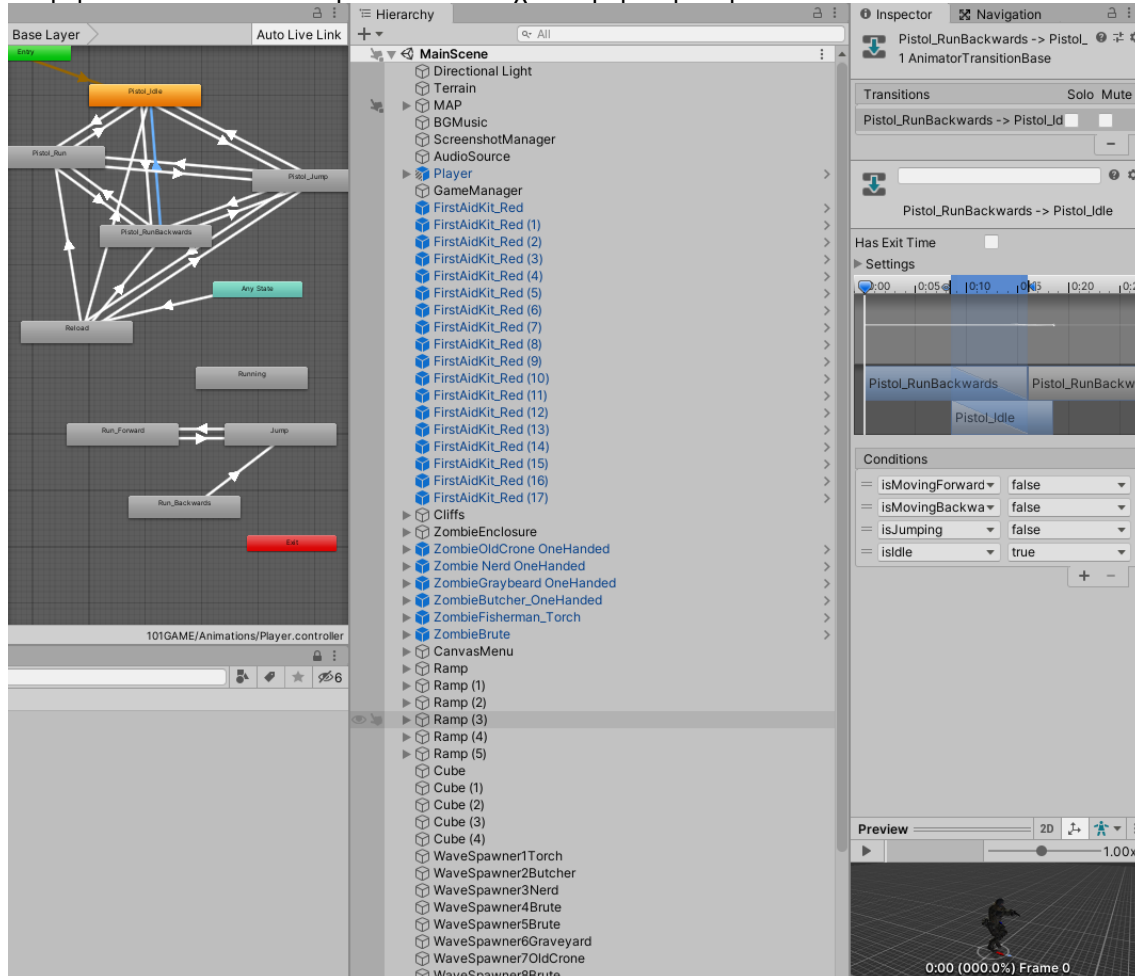


(έμφαση στο βελάκι της μετάβασής(μπλε χρώμα), αλλά και στα conditions στο inspector δεξιά)

Πτυχιακή Διατριβή

Σκαρλής Θρασύβουλος

Εδώ φαίνεται άλλο ένα ιδιαίτερα σύνθετο παράδειγμα μετάβασης καθώς χρειάζεται τέσσερα διαφορετικά Parameters για να επιτευχθεί η μετάβαση τα οποία είναι όλα Boolean.



(έμφαση στο βελάκι της μετάβασης(μπλε χρώμα), αλλά και στα conditions στο inspector δεξιά)

Base Layer Animation Zombies

Για το Animator των zombies χρησιμοποιήθηκε μόνο ένα layer, το base layer, και αυτό γιατί οτιδήποτε παραπάνω απλά θα ανέβαζε την πολυπλοκότητα παρά θα βοηθούσε. Όπως και για τον παίχτη έτσι και εδώ δεν χρησιμοποιήθηκε το root motion, καθώς όλα ελέγχονται μέσω κώδικα που θα αναλυθεί στο επόμενο κεφάλαιο.

Στην παρακάτω εικόνα παρατηρούμε πως δεν χρησιμοποιούνται καθόλου δυο animation states που παρέχονται από τον δημιουργό των asset(δεν υπάρχουν βελάκια προς και από αυτά).

Τα zombies έχουν πέντε διαφορετικά states/animations,(1. Idle, 2. Walk ή Run ανάλογα με την ταχύτητα που κινούνται, 3. OneHanded1_Attack_1,4. OneHanded1_Attack2 και 5. Death).

Επίσης έχουν τα εξής parameters: Boolean= isRunning, isAttacking, isDead, Triggers= Attack1, Attack2, Die και Attack.

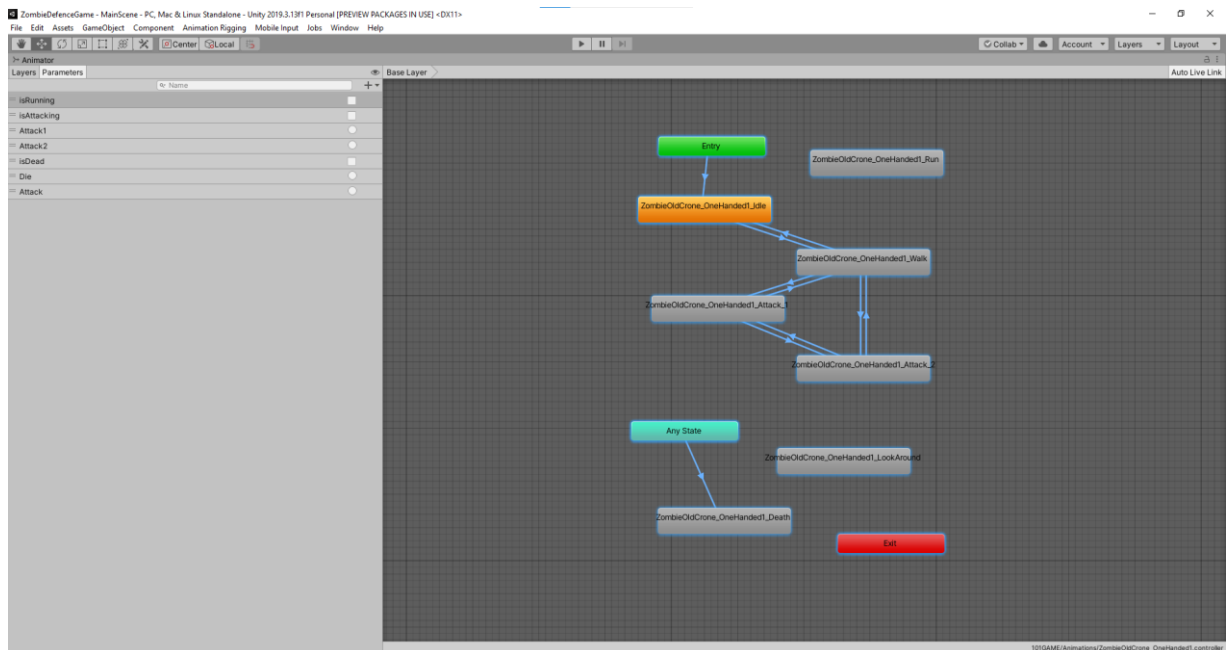
Ασφαλώς και εδώ παρατηρείται πως είναι παρόν τα states entry, Any state και Exit. Αυτή τη φορά το Any state οδηγεί στο Death state/animation, καθώς όπου και να βρίσκεται το ζόμπι, όσον αφορά τα animations πρέπει όταν χάνει όλο το HP του να μεταβεί στο death animation.

Η λογική στο base layer του ζόμπι είναι πολύ απλούστερη από αυτή του παίχτη, για αυτόν τον λόγο υπάρχουν λιγότερα βέλη/μεταβάσεις συνολικά. Συγκεκριμένα το κάθε ζόμπι ξεκινάει από ένα idle state και κατευθείαν αλλάζει σε κάποιο walking ή running. Στη συνέχεια ανάλογα με τα

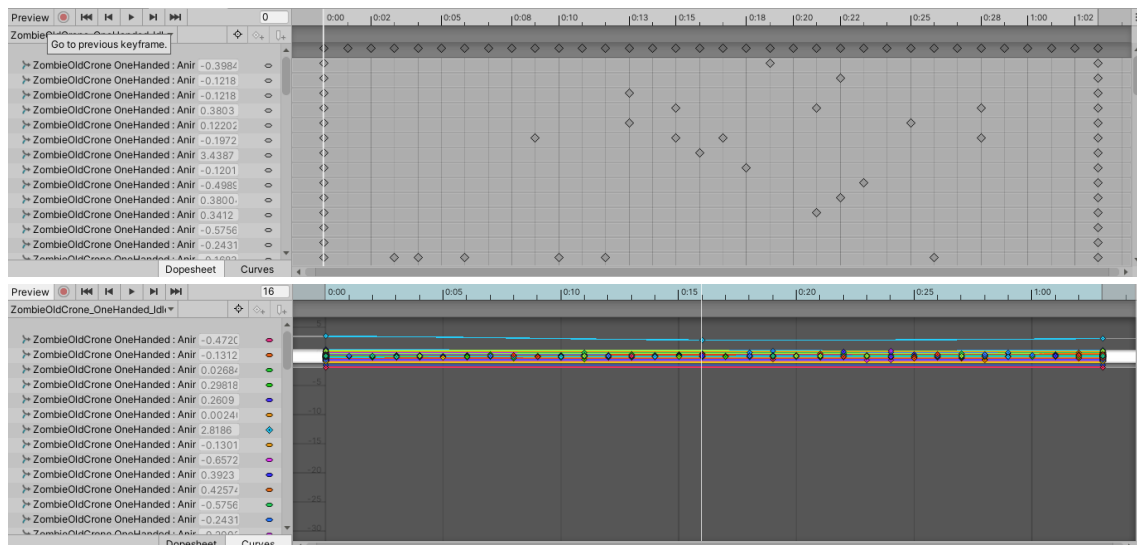
Πτυχιακή Διατριβή

Σκαρλής Θρασύβουλος

parameters θα εκτελέσει το πρώτο attack animation και μετά το δεύτερο. Έπειτα είτε θα συνεχίσει να εκτελεί τα attack animations διαδοχικά, είτε θα επιστρέψει σε walking/running animation.



(Παράδειγμα Animator για το Zombie OldCrone)



7 Scripts

7.1 Εισαγωγή στα Scripts

Παρακάτω ονοματικά τα scripts και μια μικρή περιγραφή αυτών :

- **EnemySpawner** υπεύθυνο για να εμφανίζει τα διάφορα ζόμπι στο χάρτη.
- **BulletProjectile** υπεύθυνο για τη συμπεριφορά της σφαίρας που ρίχνει το όπλο και τα “blood particles” δηλαδή του αίμα που βγάζει τα ζόμπι όταν τα πετυχαίνουν σφαίρες.
- **ScreamSound** υπεύθυνο να παίζει ένα ήχο σε τυχαία χρονικά διαστήματα και να “buffάρει” τα zombies.
- **ZombieDeathHandler** υπεύθυνο για το θάνατο του ειδικού “torch” ζόμπι.
- **ZombieController** υπεύθυνο για τις περισσότερες από τις συμπεριφορές των ζόμπι.
- **ZombieHealth** υπεύθυνο για τη ζωή των ζόμπι και τα animations.
- **PlayerBehavior** υπεύθυνο για τις περισσότερες από τις συμπεριφορές του παίχτη.
- **PlayerMovement** υπεύθυνο για τις περισσότερες από τις συμπεριφορές του παίχτη.
- **EndMenu** υπεύθυνο το φόρτωμα των σκηνών
- **FirstPersonCamera** υπεύθυνο για τη κίνηση της κάμερας του παιχνιδιού
- **HealthPack** υπεύθυνο για τα healthpacks
- **MainMenu** υπεύθυνο για τη πρώτη σκηνή του παιχνιδιού
- **MoveReticle** υπεύθυνο για τον στόχο του παίχτη
- **PauseMenu** υπεύθυνο για το μενού όταν πατιέται το ESC
- **ScreenshotDisplay** υπεύθυνο για το screenshot
- **ScreenshotManager** υπεύθυνο να κρατήσει το τελευταίο frame του παιχνιδιού
- **Timer**
- **ZombieDeathTracker** υπεύθυνο για το ποια και πόσα ζόμπι έχει σκοτώσει ο παίχτης.

7.2 Βασικά Scripts των zombies

Το script **ZombieController** διαχειρίζεται τη συμπεριφορά κάθε zombie που ακολουθεί έναν στόχο(τον παίχτη), επιτίθεται σε αυτόν όταν βρίσκεται εντός εμβέλειας, και μπορεί να δεχτεί προσωρινά ενισχυμένα στατιστικά (buff). Το script χρησιμοποιεί το NavMeshAgent για την πλοήγηση και το animator για τον έλεγχο των animations.

Στη συνέχεια ακολουθεί λεπτομερής επεξήγηση του κώδικα.

```

1  using System.Collections;
2  using UnityEngine;
3  using UnityEngine.AI;
4
5  [UnityScript 1.2 references]
6  public class ZombieController : MonoBehaviour
7  {
8      private NavMeshAgent agent = null;
9      private Animator anim = null;
10     [SerializeField] private Transform target;
11
12     // Distance at which the zombie will start attacking
13     [SerializeField] private float attackRange = 2.0f;
14     [SerializeField] private float originalSpeed; // Original movement speed (to be set from NavMeshAgent)
15     private float speed;
16     private bool isBuffed = false;
17     private float buffEndTime = 0f;
18
19     // Buff parameters
20     [SerializeField] private float buffSpeedMultiplier = 4f; // Speed multiplier during the buff
21     [SerializeField] private float additionalDamage = 15f; // Additional damage during the buff
22
23     // Refocus parameters
24     [SerializeField] private float refocusInterval = 5f; // Time in seconds between refocuses
25     private float refocusTimer = 0f;
26

```

UnityEngine.AI: Παρέχει εργαλεία για πλοήγηση (AI navigation) μέσω του NavMeshAgent.

NavMeshAgent agent: Χρησιμοποιείται για την πλοήγηση του zombie στο χώρο.

Animator anim: Χρησιμοποιείται για τον έλεγχο των animations (π.χ. τρέξιμο, επίθεση).

Τι είναι το refocus;

Μια τεχνική που χρησιμοποιήθηκε για την αρμονική κίνηση των zombie, πάντα με στόχο τον παίχτη, είναι αυτή του refocus, με την οποία διασφαλίζεται πως όποιο πρόβλημα και αν προκύψει στην κινησιολογία των zombie θα λυθεί μετά από λίγα δευτερόλεπτα καθώς το ζόμπι επανατοποθετεί τον παίχτη ως στόχο και ξεκινάει το «ταξίδι» του προς αυτόν από την αρχή. Τα προβλήματα που αποφεύγονται με αυτόν τον τρόπο, είναι τυχόν προβλήματα που μπορεί να προκύψουν καθώς το zombie, έρχεται σε επαφή με, ελλατωματικά assets ή και μεμονωμένα μέρη αυτών, με αποτέλεσμα να μην γίνεται σωστή χρήση του NavMeshAgent.

refocusInterval: Το διάστημα χρόνου σε δευτερόλεπτα για την ανανέωση του στόχου.

refocusTimer: Timer που παρακολουθεί το πότε πρέπει να γίνει refocus.

```

27 void Start()
28 {
29     agent = GetComponent<NavMeshAgent>();
30     anim = GetComponent<Animator>();
31
32     // Initialize speed
33     originalSpeed = agent.speed; // Set originalSpeed from NavMeshAgent speed
34     speed = originalSpeed;
35     refocusTimer = refocusInterval; // Initialize refocus timer
36 }
37
38 @ Unity Message | 0 references
39 void Update()
40 {
41     MoveToTarget();
42     HandleBuffExpiration();
43     HandleRefocus();
44 }
45
46 // Function to be called when the enemy starts moving
47 1 reference
48 public void StartMoving()
49 {
50     anim.SetBool("isRunning", true);
51     anim.SetBool("isAttacking", false);
52 }
53
54 // Function to be called when the enemy stops moving
55 1 reference
56 public void StopMoving()
57 {
58     anim.SetBool("isRunning", false);
59     anim.SetBool("isAttacking", false);
60     agent.destination = transform.position;
61 }

```

Οι μέθοδοι :

Start: Αρχικοποιεί τις βασικές παραμέτρους (NavMeshAgent, Animator, speed) και ρυθμίζει το timer για το refocus.

Update: Ενημερώνει συνεχώς τη συμπεριφορά του zombies

- MoveToTrget(): κάνει τα zombies να κινούνται προς τον παίχτη.
- HandleBuffExpiration(): Ελέγχει αν το buff έχει λήξει.
- HandleRefocus(): Ανανεώνει την πορεία προς τον παίχτη σε τακτά διαστήματα.

StartMoving και StopMoving: ενεργοποιεί και αντίστοιχα σταματά το animation για την κίνηση.

```
60 // Function to be called when the enemy is attacking
61 1 reference
62 public void Attack()
63 {
64     int randomIndex = Random.Range(1, 3); // Choose between 1 and 2
65     string triggerName = "Attack" + randomIndex;
66     anim.SetTrigger(triggerName);
67     anim.SetBool("isAttacking", true);
68     anim.SetBool("isRunning", false);
69 }
70 1 reference
71 private void MoveToTarget()
72 {
73     agent.SetDestination(target.position);
74     float distanceToTarget = Vector3.Distance(transform.position, target.position);
75     if (distanceToTarget <= attackRange)
76     {
77         StopMoving();
78         Attack();
79     }
80     else
81     {
82         StartMoving();
83     }
84     RotateToTarget();
85 }
86 1 reference
87 private void RotateToTarget()
88 {
89     Vector3 direction = target.position - transform.position;
90     Quaternion rotation = Quaternion.LookRotation(direction, Vector3.up);
91     transform.rotation = rotation;
92 }
```

Attack: Επιλέγει τυχαία ένα από δύο animations επίθεσης.

MoveToTarget: η μέθοδος αυτή κάνει τα zombies να κινούνται προς τον παίχτη χρησιμοποιώντας το NavMesh component του Unity, καθώς επίσης λέει στα zombie να σταματήσουν να κινούνται και να επιτεθούν όταν βρίσκονται κοντά στον παίχτη, αλλά και να σταματήσουν να επιτίθενται άμα ο παίχτης απομακρυνθεί αρκετά.

RotateToTarget: υπεύθυνο για τον σωστό προσανατολισμό των zombies, δηλαδή να κινούνται και να κοιτάνε προς την ίδια κατεύθυνση. (προς τον παίχτη)

```

92
93 // Apply buff to the zombie
94 public void ApplyBuff(float duration)
95 {
96     if (!isBuffed)
97     {
98         originalSpeed = speed; // Save current speed as original speed if not already buffed
99     }
100     speed = originalSpeed * buffSpeedMultiplier;
101     isBuffed = true;
102     buffEndTime = Time.time + duration;
103
104     Debug.Log($"Buff Applied: Speed = {speed}, Buff Duration = {duration}");
105
106     // Apply the buff to zombies
107     ZombieHealth healthComponent = GetComponent<ZombieHealth>();
108     if (healthComponent != null)
109     {
110         healthComponent.ApplyBuff(additionalDamage, duration);
111     }
112 }
113
114 // Remove buff from the zombie
115 void RemoveBuff()
116 {
117     speed = originalSpeed;
118     isBuffed = false;
119
120     Debug.Log("Buff Removed");
121 }
122

```

Η παραπάνω εικόνα δείχνει πώς τα zombies αποχτούν και πώς χάνουν το buff αντίστοιχα. Αλλά και πως αυτό εφαρμόζεται. Όλες οι σταθερές τιμές είναι serialized fields έτσι ώστε να μπορούμε να τα πειράξουμε στον inspector μέσα από το Unity.

```

123 // Handle buff expiration
124 private void HandleBuffExpiration()
125 {
126     if (isBuffed && Time.time >= buffEndTime)
127     {
128         RemoveBuff();
129     }
130 }
131
132 // Handle refocus logic
133 private void HandleRefocus()
134 {
135     refocusTimer -= Time.deltaTime;
136     if (refocusTimer <= 0f)
137     {
138         // Reset the destination and target
139         agent.destination = transform.position; // Stop moving
140         agent.SetDestination(target.position); // Re-set the target
141         refocusTimer = refocusInterval; // Reset the refocus timer
142     }
143 }
144
145 // Getters for speed (assuming other parts of your code use these)
146 public float GetMovementSpeed() => speed;
147 public void SetMovementSpeed(float newSpeed) => speed = newSpeed;
148 }
149

```

Οι γραμμές 146 και 147 Εξασφαλίζουν την πρόσβαση στην (speed)ταχύτητα από άλλες κλάσεις.

Το script **ZombieHealth** διαχειρίζεται τις παραμέτρους (Health)υγείας και (damage)ζημιάς ενός zombie. Επίσης, περιλαμβάνει τη δυνατότητα να επιτεθεί στον παίκτη, να δεχτεί ζημιά, και να πεθάνει. Για όλες τις προηγούμενες ενέργειες παίζει και το ανάλογο animation. Το script υποστηρίζει την εφαρμογή προσωρινών ενισχύσεων (buffs) που αυξάνουν τη ζημιά(damage) του zombie. Επίσης επικοινωνεί με άλλο script υπεύθυνο για να σημειώνονται όλα τα kills του παίκτη, αλλά και με το προηγούμενο script που αναλύθηκε, για την άντληση δεδομένων όπως πχ αν είναι ενεργό το buff των zombies.

```

1  using System.Collections;
2  using UnityEngine;
3
4  @ UnityScript | 6 references
5  public class ZombieHealth : MonoBehaviour
6  {
7      [SerializeField] private int maxHealth = 100;
8      [SerializeField] private int baseDamage = 10; // Base damage
9      [SerializeField] private float attackRange = 3f; // Range within which the zombie can attack the player
10     private int currentHealth;
11     private Animator animator;
12     private bool isDead = false;
13     private Transform playerTransform;
14     private PlayerBehaviour playerBehaviour;
15     private bool isAttacking = false; // Flag to indicate if the zombie is currently attacking
16
17     // Buff parameters
18     private int buffedDamage;
19     private bool isBuffed = false;
20     private float buffEndTime;
21
22     void Awake()
23     {
24         animator = GetComponent<Animator>();
25         if (animator == null)
26         {
27             Debug.LogError("Animator not found on " + gameObject.name);
28         }
29
30         playerTransform = GameObject.FindGameObjectWithTag("Player").transform;
31         if (playerTransform == null)
32         {
33             Debug.LogError("Player transform not found for " + gameObject.name);
34         }
35         else
36         {
37             playerBehaviour = playerTransform.GetComponent<PlayerBehaviour>();
38             if (playerBehaviour == null)
39             {
40                 Debug.LogError("PlayerBehaviour component not found on player.");
41             }
42         }
43     }
44
45     @ UnityMessage | 0 references
46     void Start()
47     {
48         currentHealth = maxHealth;
49         StartCoroutine(AttackRoutine());
50     }

```

Ορισμένες στατιστικά που αφορούν τα zombie(health,damage,attackRange etc).
playerBehaviour: Αναφορά στο script του παίκτη για την κλήση μεθόδων όπως η TakeDamage.

Awake: Αρχικοποιεί τις βασικές εξαρτήσεις:

- Animator: Ελέγχει τα animations.
- playerTransform: Εντοπίζει τη θέση του παίκτη μέσω tag.

playerBehaviour: Αναφορά στο script του παίκτη για την κλήση μεθόδων.

Λίγα λόγια για τα coroutines:

Τα coroutines είναι ένα ισχυρό εργαλείο στον προγραμματισμό με το Unity και χρησιμοποιούνται για τη δημιουργία συμπεριφορών που εκτελούνται με διακοπές σε διαφορετικά χρονικά διαστήματα, χωρίς να μπλοκάρουν την κύρια ροή εκτέλεσης του προγράμματος. Είναι ιδιαίτερα χρήσιμες για προγραμματισμό ασύγχρονων λειτουργιών, όπως η αναμονή για ένα συμβάν, η εκτέλεση αργών λειτουργιών ή η δημιουργία χρονισμένων εφέ.

Στο Unity, οι coroutines είναι ειδικές μέθοδοι που μπορούν να σταματούν προσωρινά την εκτέλεσή τους και να συνεχίζουν σε επόμενο καρέ ή μετά από συγκεκριμένο χρόνο.

Χρησιμοποιούνται με το `StartCoroutine` και συνήθως επιστρέφουν έναν τύπο `IEnumerator`.

Πλεονεκτήματα:

1. Ευκολία: Παρέχουν απλό και κατανοητό τρόπο χειρισμού χρονοκαθυστερήσεων ή ακολουθιακών συμβάντων.
2. Ασύγχρονες Λειτουργίες: Δεν μπλοκάρουν το κύριο νήμα (main thread), διατηρώντας το παιχνίδι ομαλό.
3. Αναγνωσιμότητα: Οι κώδικες με καθυστερήσεις γίνονται πιο ευανάγνωστοι και οργανωμένοι.

```
50  IEnumerator AttackRoutine()
51  {
52      while (true)
53      {
54          yield return new WaitForSeconds(2f); // Adjust attack frequency as needed
55          if (!isDead && !isAttacking)
56          {
57              StartCoroutine(Attack());
58          }
59      }
60  }
61
62  IEnumerator Attack()
63  {
64      if (isAttacking) yield break; // Exit if already attacking
65
66      isAttacking = true;
67      animator.SetTrigger("Attack");
68
69      // Wait for a short time for the attack animation to progress
70      yield return new WaitForSeconds(0.5f);
71
72      // Perform raycast from zombie towards player
73      RaycastHit hit;
74      if (Physics.Raycast(transform.position, (playerTransform.position - transform.position).normalized, out hit, attackRange))
75      {
76          if (hit.collider.CompareTag("Player"))
77          {
78              //Debug.Log("Zombie hit player!");
79              playerBehaviour.TakeDamage(GetDamage());
80          }
81      }
82
83      isAttacking = false;
84  }
85
```

AttackRoutine: Επαναλαμβανόμενα καλεί τη μέθοδο `Attack` κάθε 2 δευτερόλεπτα, ελέγχοντας πάντα αν το zombie είναι νεκρό ή αν είδη επιτίθεται.

Attack:

- **TakeDamage:** Καλεί τη μέθοδο ζημιάς του παίκτη με την τρέχουσα ζημιά του zombie.
- **Physics.Raycast:** Ελέγχει αν ο παίκτης βρίσκεται εντός εμβέλειας επίθεσης, στέλνοντας μια ακτίνα από το χέρι του zombie και υπολογίζει άμα ο παίκτης ήταν εντός εμβέλειας της επίθεσης, ώστε να καλέσει τη μέθοδο `TakeDamage` και ο παίκτης να φάει το ανάλογο `damage`(ζημιά).

```
86 1 reference  
87 public void TakeDamage(int amount)  
88 {  
89     if (isDead) return; // Ensure no damage is taken if the zombie is dead  
90  
91     currentHealth -= amount;  
92     if (currentHealth <= 0)  
93     {  
94         Die();  
95     }  
96 }  
97 1 reference  
98 private void Die()  
99 {  
100     if (isDead) return; // Ensure this method is called only once  
101  
102     isDead = true;  
103     animator.SetTrigger("Die");  
104     //Debug.Log("Zombie died.");  
105  
106     // Notify the ZombieDeathTracker  
107     ZombieDeathTracker.Instance?.RegisterZombieDeath(gameObject.name);  
108  
109     // Additional death logic (e.g., disable collider, stop movement, etc.)  
110     Destroy(gameObject, 4.5f); // Destroy after 4.5 seconds to allow death animation to play  
111 }  
112 1 reference  
113 public bool IsDead()  
114 {  
115     return isDead;  
116 }
```

TakeDamage: Μειώνει την υγεία του zombie. Αν η υγεία γίνει μηδέν, καλείται η μέθοδος Die.

Die: Αυτή η μέθοδος καλείται μόνο μια φορά, και αφού δώσει περίπου 5 δευτερόλεπτα χρόνο στο zombie για να κάνει το death animation (το οποίο καλεί), τότε καταστρέφει το zombie και ενημερώνει σχετική μέθοδο (ZombieDeathTracker) ώστε να καταγραφεί το kill του παίχτη.

```

117 // Apply buff to increase damage
118 1 reference
119 public void ApplyBuff(float additionalDamage, float duration)
120 {
121     if (!isBuffed)
122     {
123         buffedDamage = baseDamage + (int)additionalDamage;
124     }
125     else
126     {
127         buffedDamage = Mathf.Max(buffedDamage, baseDamage + (int)additionalDamage);
128     }
129     isBuffed = true;
130     buffEndTime = Time.time + duration;
131     Debug.Log($"Buff Applied: Damage = {buffedDamage}, Buff Duration = {duration}");
132 }
133
134 // Remove buff from the zombie
135 1 reference
136 void RemoveBuff()
137 {
138     isBuffed = false;
139     buffedDamage = baseDamage;
140     Debug.Log("Buff Removed");
141 }
142
143 // Get damage considering buff
144 1 reference
145 public int GetDamage()
146 {
147     if (isBuffed && Time.time >= buffEndTime)
148     {
149         RemoveBuff();
150     }
151     return isBuffed ? buffedDamage : baseDamage;
152 }
153

```

ApplyBuff: Αυξάνει τη ζημιά του zombie για συγκεκριμένη διάρκεια.

RemoveBuff: Επαναφέρει τη βασική ζημιά.

GetDamage: Επιστρέφει την τρέχουσα ζημιά, ελέγχοντας αν το buff έχει λήξει.

7.3 Βασικά Scripts του παίκτη

Ακολουθεί η αναλυτική ανάλυση του script **PlayerBehaviour**, που διαχειρίζεται τη ζωή, το επίπεδο, την εμπειρία (XP), και άλλες λειτουργίες που σχετίζονται με τον παίκτη. Το script PlayerBehaviour ελέγχει τη συμπεριφορά του παίκτη, περιλαμβάνοντας:

- Διαχείριση υγείας (λήψη ζημιάς, θεραπεία, ενημέρωση UI).
- Λειτουργίες επιπέδου και εμπειρίας (leveling system).
- Οπτική ανατροφοδότηση ζημιάς μέσω overlay.
- Αντίδραση στο θάνατο του παίκτη.

Στην παρακάτω εικόνα, στην κορυφή φαίνονται ορισμένα πράγματα απαραίτητα για το UI. Στη συνέχεια μερικές σταθερές από τα στατιστικά του παίκτη (maxHealth, level etc). Η γραμμή 23 παρέχει αναφορά στο script κίνησης του παίκτη (PlayerMovement), ώστε να αναβαθμίζονται οι παράμετροι κίνησης κατά την άνοδο επιπέδου.

```

@ Unity Script | 9 references
public class PlayerBehaviour : MonoBehaviour
{
    [SerializeField] private Image currentHealthFill;
    [SerializeField] private Text currentHealthText;
    [SerializeField] private Image damageOverlay;
    [SerializeField] private float overlayDuration = 0.5f;
    [SerializeField] private Color overlayColor = new Color(1, 0, 0, 0.5f);
    [SerializeField] private TextMeshProUGUI levelText;
    [SerializeField] private TextMeshProUGUI currentXPText;

    private int currentHealth;
    public int maxHealth { get; private set; } = 100; // Define maxHealth with a getter

    private int currentXP;
    private int level = 1;
    private Coroutine damageCoroutine;

    [SerializeField] private PlayerMovement playerMovement;

    private void Start()
    {
        currentHealth = maxHealth;
        UpdateHealthUI();
        UpdateLevelUI();

        if (damageOverlay != null)
        {
            damageOverlay.color = Color.clear;
        }
        else
        {
            Debug.LogError("Damage overlay is not assigned.");
        }

        currentXPText.text = currentXP.ToString();
    }
}

```

Start: Καλεί τις UpdateHealthUI και UpdateLevelUI για ενημέρωση της γραφικής διεπαφής. Επίσης εξασφαλίζει ότι το overlay είναι μηδενισμένο αρχικά.

```

// Function to add XP to the player
public void AddXP(int xpAmount)
{
    int xpToAdd = Mathf.FloorToInt(xpAmount * GetXPMultiplier());
    currentXP += xpToAdd;

    if (currentXP >= 100)
    {
        level++;
        currentXP -= 100;
        UpdateLevelUI();
        playerMovement.LevelUpStats(level); // Call LevelUpStats in PlayerMovement
    }

    currentXPText.text = currentXP.ToString();
}

```

AddXP: Αυξάνει το XP και αναλόγως ενημερώνει και το αντίστοιχο UI

```
43 // Function to apply damage to the player
44 1 reference
45 public void TakeDamage(int damageAmount)
46 {
47     currentHealth -= damageAmount;
48     if (currentHealth < 0)
49     {
50         currentHealth = 0;
51     }
52     UpdateHealthUI();
53     if (currentHealth <= 0)
54     {
55         HandlePlayerDeath();
56     }
57     if (damageCoroutine != null)
58     {
59         StopCoroutine(damageCoroutine);
60     }
61     damageCoroutine = StartCoroutine(ShowDamageOverlay());
62 }
63
64
65 // Method to handle player death
66 1 reference
67 private void HandlePlayerDeath()
68 {
69     // Disable the PauseMenu through GameManager
70     GameManager.gamemanager.DisablePauseMenu();
71     // Load the defeat scene or handle player death logic
72     GameManager.gamemanager.LoadDefeatScene();
73 }
74
```

TakeDamage: Μειώνει την τρέχουσα υγεία. και Εμφανίζει το overlay ζημιάς μέσω coroutine (ShowDamageOverlay).. Αν γίνει μηδενική, καλείται η HandlePlayerDeath.

HandlePlayerDeath: απενεργοποιεί το pausemenu και μέσω του gamemanager φορτώνει την τελική σκηνή του παιχνιδιού (DefeatScene).

```

75 // Function to heal the player
76 public void Heal(int healAmount)
77 {
78     currentHealth += healAmount;
79     if (currentHealth > maxHealth)
80     {
81         currentHealth = maxHealth;
82     }
83     UpdateHealthUI();
84 }
85
86 // Function to update the UI elements
87 private void UpdateHealthUI()
88 {
89     float fillAmount = (float)currentHealth / maxHealth;
90     currentHealthFill.fillAmount = fillAmount;
91     currentHealthText.text = currentHealth.ToString();
92 }
93
94 // Coroutine to show the damage overlay
95 private IEnumerator ShowDamageOverlay()
96 {
97     if (damageOverlay != null)
98     {
99         damageOverlay.color = overlayColor;
100
101         yield return new WaitForSeconds(overlayDuration);
102
103         damageOverlay.color = Color.clear;
104     }
105     else
106     {
107         Debug.LogError("Damage overlay is not assigned.");
108     }
109 }

```

Heal: αναπληρώνει λίγο HP στον παίχτη

UpdateHealthUI: υπεύθυνη να μεταβάλει την μπάρα του HP ανάλογα με το πόσο HP έχει ο παίχτης, αλλά και να ενημερώνει την τιμή της.

ShowDamageOverlay: ουσιαστικά κάθε φορά που ο παίχτης τρώει damage, κάνει την οθόνη να κοκκινίζει.

Το script **PlayerMovement** είναι υπεύθυνο για τον έλεγχο της κίνησης, της σωματικής δράσης και της μάχης του παίκτη. Υλοποιεί διάφορες λειτουργίες, όπως η μετακίνηση και το τρέξιμο, το άλμα, η βολή σφαιρών, η φόρτωση πυρομαχικών (reload) και η αναβάθμιση των στατιστικών του παίκτη καθώς ανεβαίνει level. Παράλληλα, χειρίζεται την ενημέρωση του UI για τις σφαίρες και είναι παράλληλα υπεύθυνο για την σωστή εκτέλεση κάποιων εκ' των animation. Λόγω του μεγάλου όγκου αυτού του script θα αναλυθούν μόνο επιλεγμένα κομμάτια του.

Όπως φαίνεται στην παρακάτω εικόνα, για την κίνηση του παίκτη χρησιμοποιήθηκε το νέο input system του Unity. Το Input System του Unity είναι ένα προηγμένο σύστημα διαχείρισης

Πτυχιακή Διατριβή

Σκαρλής Θρασύβουλος

εισόδου, σχεδιασμένο να προσφέρει μεγαλύτερη ευελιξία, συνέπεια και δυνατότητες σε σχέση με το παλαιότερο (legacy) Input Manager. Χρησιμοποιείται για τη διαχείριση εισόδων από πληκτρολόγια, ποντίκια, χειριστήρια, οθόνες αφής, ακόμα και από συσκευές VR/AR.

Το σύστημα οργανώνεται γύρω από τα "Input Actions", τα οποία αντιπροσωπεύουν συγκεκριμένες ενέργειες (όπως "Jump", "Move", "Shoot"). Οι ενέργειες αυτές μπορούν να αντιστοιχιστούν σε συγκεκριμένα πλήκτρα ή συνδυασμούς εισόδων.

Υποστηρίζει διαφορετικές συμπεριφορές για εισόδους όπως "Press", "Hold", και "Release", κάνοντας εύκολο τον προγραμματισμό πολύπλοκων μηχανισμών.

Το νέο Input System πρέπει να εγκατασταθεί μέσω του Unity Package Manager και να ενεργοποιηθεί στις ρυθμίσεις του project (Player Settings > Active Input Handling). Ο χρήστης δημιουργεί ένα asset αρχείο Input Actions, όπου ορίζονται οι ενέργειες (actions) και οι συσχετισμένες εισόδους (bindings).

Κάτι αντίστοιχο έχει γίνει και στο παιχνίδι μας όπως φαίνεται στην από κάτω εικόνα.

```
7 public class PlayerMovement : MonoBehaviour
8 {
9     PlayerInput playerInput;
10    InputAction moveAction;
11    InputAction jumpAction;
12    InputAction sprintAction;
13    InputAction reloadAction;
14    InputAction shootAction;
15
```

Όπως βλέπουμε στην παρακάτω εικόνα οι μέθοδοι Start και Update υπάρχουν αφού είναι απαραίτητοι για τέτοιου είδους script.

```
52 void Start()
53 {
54     playerInput = GetComponent<PlayerInput>();
55     moveAction = playerInput.actions.FindAction("Move");
56     jumpAction = playerInput.actions.FindAction("Jump");
57     sprintAction = playerInput.actions.FindAction("Sprint");
58     reloadAction = playerInput.actions.FindAction("Reload");
59     shootAction = playerInput.actions.FindAction("Shoot");
60     rb = GetComponent<Rigidbody>();
61     anim = GetComponent<Animator>();
62
63     currentSpeed = defaultSpeed;
64     lastInputTime = Time.time;
65     navMeshAgent = GetComponent<NavMeshAgent>();
66     rb.freezeRotation = true;
67     shootAction.performed += _ => Shoot();
68
69     currentAmmo = maxAmmo;
70     UpdateAmmoUI();
71 }
72
73 void Update()
74 {
75     MovePlayer();
76     CheckJump();
77     CheckSprint();
78     CheckReload();
79     CheckIdle();
80     screenCenterWorldPosition = GetCenterScreenWorldPosition();
81 }
82
```

Η Start είναι υπεύθυνη για την αρχικοποίηση ελέγχων. Συνδέει τις ενέργειες από το PlayerInput με τις ενέργειες του παίκτη. Ενημερώνει του UI την απεικόνιση των πυρομαχικών(ammo) κατά την έναρξη.

Η μέθοδος Update στο script **PlayerMovement** εκτελείται σε κάθε καρτέ (frame) του παιχνιδιού και αποτελεί την "καρδιά" του script, καθώς συντονίζει όλες τις βασικές λειτουργίες που σχετίζονται με τη συμπεριφορά του παίκτη. Περιέχει κλήσεις προς επιμέρους μεθόδους, οι οποίες ελέγχουν τη συμπεριφορά του παίκτη ανάλογα με τις εισόδους (inputs) και την κατάσταση του παιχνιδιού.

- MovePlayer(); // Χειρίζεται την κίνηση του παίκτη.
- CheckJump(); // Ελέγχει αν ο παίκτης πατάει το κουμπί για άλμα.
- CheckSprint(); // Ελέγχει αν ο παίκτης πατάει το κουμπί για τρέξιμο.
- CheckReload(); // Ελέγχει αν ο παίκτης προσπαθεί να φορτώσει πυρομαχικά.
- CheckIdle(); // Ελέγχει αν ο παίκτης είναι ανενεργός (idle).
- screenCenterWorldPosition = GetCenterScreenWorldPosition(); // Υπολογίζει τη θέση στο κέντρο της οθόνης για χρήση στη στόχευση.

Η μέθοδος Update Διαχειρίζεται την κίνηση, τρέξιμο, άλμα και μάχη του παίκτη σε πραγματικό χρόνο. Συνδυάζει μηχανισμούς αλληλεπίδρασης με το περιβάλλον (π.χ., στόχευση). Συνδυάζει μηχανισμούς αλληλεπίδρασης με το περιβάλλον (π.χ., στόχευση).

```

144 void Jump()
145 {
146     navMeshAgent.enabled = false;
147     rb.velocity = Vector3.zero;
148     rb.AddForce(Vector3.up * jumpPower, ForceMode.Impulse);
149     isJumping = true;
150     anim.SetTrigger("Jump");
151     StartCoroutine(ResetJumpState());
152 }
153

```

Μία μέθοδος που αξίζει να σχολιαστεί είναι η Jump καθώς για να επιτευχθεί το τέλειο άλμα, χρειάστηκε να απενεργοποιηθεί τελείως η ταχύτητα που παίρνει το asset του παίχτη (δηλ. το μοντέλο) σε έναν άξονα, με σκοπό να μην πέφτει προς τα εμπρός(δηλ. με το πρόσωπο στο πάτωμα) και ταυτόχρονα να απενεργοποιηθεί και το navMesh component για να επιτρέψει στο μοντέλο να μην έχει συνεχόμενη επαφή με το πάτωμα.

```

189 IEnumerator ResetJumpState()
190 {
191     yield return new WaitForSeconds(1f);
192     isJumping = false;
193     navMeshAgent.enabled = true;
194 }
195

```

Ο τρόπος που ο παίχτης δεν μπορεί να πηδήξει στον αέρα είναι ξανά είναι πολύ απλός με ένα timer/delay, ο χρήστης δεν έχει τη δυνατότητα να ξαναπηδήξει.

```

1 reference
97 void MovePlayer()
98 {
99     // Get the forward direction of the camera, but ignore the y-axis (up/down direction) to prevent the player from tilting
100     Vector3 cameraForward = Vector3.Scale(Camera.main.transform.forward, new Vector3(1, 0, 1)).normalized;
101
102     // Calculate the movement direction based on the camera's forward direction and the right direction,
103     // multiplied by the player's input from the keyboard/controller (moveAction)
104     Vector3 moveDirection = cameraForward * moveAction.ReadValue<Vector2>().y + Camera.main.transform.right * moveAction.ReadValue<Vector2>().x;
105
106     // Determine the current speed; if the player is sprinting, multiply the speed by the sprintSpeedMultiplier, otherwise use the normal speed
107     float currentSpeed = isSprinting ? speed * sprintSpeedMultiplier : speed;
108
109     // Update the player's position by moving in the calculated direction at the calculated speed, scaled by deltaTime to ensure frame rate independence
110     transform.position += moveDirection * currentSpeed * Time.deltaTime;
111
112     // Update animation parameters based on the player's movement input to reflect movement animations (forward, backward)
113     UpdateAnimationParameters(moveAction.ReadValue<Vector2>());
114 }
115
116

```

Η μέθοδος **MovePlayer** είναι υπεύθυνη για την κίνηση του παίκτη στον χώρο, λαμβάνοντας υπόψη την κατεύθυνση της κάμερας και τις εισόδους από τον παίκτη (πληκτρολόγιο ή χειριστήριο). Συνδυάζει τη φυσική, την προσανατολισμένη κίνηση, και την ενεργοποίηση animations

ώστε να εξασφαλίσει ομαλή εμπειρία κίνησης.

Ακολουθεί μια αναλυτική ανάλυση της μεθόδου:

A. Υπολογισμός Κατεύθυνσης Κίνησης(γραμμή100):

1. Ανάκτηση Κατεύθυνσης της Κάμερας (cameraForward):

- **Camera.main.transform.forward**: Παίρνει την κατεύθυνση προς τα εμπρός της κάμερας.
- **Vector3.Scale(Camera.main.transform.forward, new Vector3(1, 0, 1))**: Αφαιρεί το ύψος (άξονας y), ώστε η κίνηση να είναι μόνο στο επίπεδο x-z (δηλαδή στο επίπεδο του εδάφους).
- **Normalized** : Κανονικοποιεί το διάνυσμα ώστε να έχει μέγεθος (μήκος) 1. Αυτό εξασφαλίζει σταθερό μέγεθος για την κατεύθυνση.
- **Σκοπός**: Η κατεύθυνση της κίνησης ευθυγραμμίζεται με την κατεύθυνση που κοιτάει η κάμερα.

2. Υπολογισμός Τελικής Κατεύθυνσης Κίνησης (moveDirection)(γραμμή 104):

- **moveAction.ReadValue<Vector2>()**: Διαβάζει την κατεύθυνση εισόδου από το χειριστήριο/πληκτρολόγιο ως διάνυσμα Vector2 (x για αριστερά-δεξιά, y για μπροστά-πίσω).
- **cameraForward * moveAction.ReadValue<Vector2>().y**: Πολλαπλασιάζει την κατεύθυνση της κάμερας (μπροστά-πίσω) με την τιμή y της εισόδου.
- **Camera.main.transform.right * moveAction.ReadValue<Vector2>().x**: Προσθέτει τη δεξιά κατεύθυνση της κάμερας, πολλαπλασιασμένη με την τιμή x της εισόδου.
- **Σκοπός**: Συνδυάζει τις εισόδους του παίκτη με τον προσανατολισμό της κάμερας για να υπολογίσει την κατεύθυνση που πρέπει να κινηθεί ο παίκτης.

B. Υπολογισμός Ταχύτητας(γραμμή 107):

- Αν ο παίκτης **τρέχει (isSprinting)**, η ταχύτητα (speed) πολλαπλασιάζεται με έναν πολλαπλασιαστή ταχύτητας (sprintSpeedMultiplier).
- Αν δεν τρέχει, χρησιμοποιείται η βασική ταχύτητα (speed).
- **Σκοπός:** Εφαρμόζεται η κατάλληλη ταχύτητα, ανάλογα με το αν ο παίκτης περπατά ή τρέχει.
-

C. Ενημέρωση Θέσης του Παίκτη(γραμμή 110):

- **moveDirection:** Η κατεύθυνση στην οποία πρέπει να κινηθεί ο παίκτης.
- **currentSpeed:** Η ταχύτητα κίνησης (προσαρμοσμένη αν τρέχει).
- **Time.deltaTime:** Εξασφαλίζει ότι η κίνηση είναι ανεξάρτητη από τον ρυθμό ανανέωσης των καρτέ (frame rate).
- **Σκοπός:** Μετακινεί τον παίκτη με την κατάλληλη ταχύτητα και κατεύθυνση.

D. Ενημέρωση Animations(γραμμή 113):

- Καλείται η μέθοδος **UpdateAnimationParameters** για να ενημερώσει τα animations του παίκτη, βασισμένα στις εισόδους κίνησης.
- Για παράδειγμα:
 Αν ο παίκτης κινείται μπροστά ή πίσω, ενεργοποιείται το animation για την ανάλογη κατεύθυνση.
 Αν δεν κινείται, ενημερώνεται ότι είναι ακίνητος.

Συνολική Λειτουργία

- Υπολογίζεται η κατεύθυνση κίνησης με βάση την κάμερα και τις εισόδους του παίκτη.
- Προσαρμόζεται η ταχύτητα, λαμβάνοντας υπόψη αν ο παίκτης τρέχει.
- Μετακινείται ο παίκτης στον χώρο, λαμβάνοντας υπόψη τον χρόνο (frame rate).
- Ενημερώνονται τα animations για να ταιριάζουν με την κατεύθυνση και την ταχύτητα κίνησης.

8 Μελλοντικά Implementations

Κανένα παιχνίδι δεν είναι απόλυτα τέλειο, και αυτό ισχύει ακόμα και για τα πιο δημοφιλή παιχνίδια στον κόσμο. Η ανάπτυξη ενός παιχνιδιού είναι μια συνεχής διαδικασία, όπου κάθε νέο χαρακτηριστικό ή μηχανισμός φέρνει προκλήσεις και πιθανές βελτιώσεις. Ακόμα και οι μεγαλύτερες εταιρείες ανάπτυξης παιχνιδιών, με ομάδες που αποτελούνται από δεκάδες ή και εκατοντάδες εξειδικευμένους επαγγελματίες, βασίζονται σε συνεχή ανατροφοδότηση από την κοινότητα των παικτών.

Ένα μεγάλο **player base** είναι ουσιαστικό για να αναδειχθούν πιθανά προβλήματα, κενά ή ευκαιρίες για περαιτέρω ανάπτυξη. Οι παίκτες είναι αυτοί που ζουν το παιχνίδι από πρώτο χέρι και συχνά προσφέρουν ιδέες που μπορεί να βελτιώσουν την εμπειρία. Η δυναμική της συνεργασίας μεταξύ δημιουργών και κοινού αποτελεί βασικό παράγοντα για την επιτυχία και την εξέλιξη κάθε παιχνιδιού.

Στο μέλλον, σκοπεύουμε να επικεντρωθούμε σε διάφορες βελτιώσεις, όπως:

Διόρθωση bugs και βελτίωση της σταθερότητας: Με την ανατροφοδότηση από τους παίκτες, μπορούμε να εντοπίσουμε και να διορθώσουμε θέματα που δεν παρατηρήθηκαν κατά τη φάση του testing.

Προσθήκη νέων χαρακτηριστικών: Η εισαγωγή νέων game mechanics, χαρακτήρων, όπλων ή περιβαλλόντων μπορεί να κρατήσει το ενδιαφέρον των παικτών. Συγκεκριμένα πιθανό είναι να δοκιμάζαμε να εισάγουμε και άλλο/άλλους χαρακτήρες, με διαφορετικά όπλα. Θα μπορούσε κάποιος να είχε κοντινή δράση(melee combat) πχ με κάποιο σπαθί ή μαχαίρια. Ίσως τόξα και μακριές λόγχες. Αυτοί οι χαρακτήρες θα μπορούσαν να είχαν άλλα στατιστικά(stats) πχ. πόσο γρήγορα ρίχνουν ή να κλέβουν ζωή με κάθε χτύπημα κτλ. Θα μπορούσαμε επιπλέον να βάλουμε επιπλέον εχθρούς πάρα πολύ δυνατούς (κάτι σαν boss fight). Παραπάνω πίστες (maps). Ακόμα και ίσως επιπλέον mode παιχνιδιού. Αντί endless horde mode (όπως είναι τώρα) να έχει και time mode όπου πχ θα έχει συγκεκριμένα ζόμπι, αλλά θ πρέπει να τα σκοτώσεις όσο το δυνατόν γρηγορότερα. Ακόμα και στοιχεία υποστήριξης πολλών ταυτόχρονων παιχτών (multiplayer) και ίσως και κάποια κατάταξη για κάθε διαφορετικό mode.

Εξισορρόπηση gameplay(balancing): Η συνεχής παρακολούθηση των στατιστικών του παιχνιδιού θα μας επιτρέψει να βελτιώνουμε τη δίκαιη κατανομή δυνάμεων και δυνατοτήτων.

Ακρόαση της κοινότητας: Με ειδικά forums, ερωτηματολόγια, και ανοιχτούς διαλόγους, οι προτάσεις της κοινότητας θα αποτελούν σημαντική βάση για τις μελλοντικές αλλαγές.

Η ανάπτυξη ενός παιχνιδιού δεν είναι ποτέ στατική. Είναι μια ζωντανή διαδικασία, όπου κάθε νέα ιδέα, ανακάλυψη ή τάση μπορεί να φέρει καινούριες δυνατότητες. Με την υποστήριξη της κοινότητας και τη δέσμευσή μας για ποιότητα, το παιχνίδι μας έχει κάθε δυνατότητα να εξελιχθεί σε κάτι μοναδικό και διαχρονικό.

9 Ορολογία

1. Εργαλεία Ανάπτυξης και Τεχνολογίες

- **Unity:** Game engine που χρησιμοποιήθηκε για τη δημιουργία του παιχνιδιού.
- **C#:** Γλώσσα προγραμματισμού για scripting.
- **Game engine:** Πλατφόρμα ανάπτυξης παιχνιδιών.
- **Asset store:** Αγορά για προ-κατασκευασμένα αντικείμενα, scripts, ήχους, και γραφικά.
- **Scripts:** Τμήματα κώδικα που καθορίζουν τη λογική του παιχνιδιού.
- **Animator:** Εργαλείο για τη διαχείριση animations.
- **Physics engine:** Μηχανισμός για τη ρεαλιστική φυσική αλληλεπίδραση αντικειμένων.

2. Ορολογία που αφορά το παιχνίδι

- **Survive The Zombies:** Τίτλος του παιχνιδιού.
- **Survival/Horde Mode:** Είδος παιχνιδιού με στόχο την επιβίωση σε κύματα εχθρών.
- **XP (Experience Points):** Πόντοι εμπειρίας που κερδίζει ο παίκτης.
- **Stats:** Στατιστικά του παίκτη (π.χ., ταχύτητα, υγεία, ζημιά).
- **HP (Health Points):** Ζωή του παίκτη ή των εχθρών.
- **Reload Speed:** Ταχύτητα επαναγέμισης όπλου.
- **Sprint Multiplier:** Πολλαπλασιαστής ταχύτητας τρεξίματος.
- **Level-Up Mechanics:** Σύστημα αναβάθμισης επιπέδου.
- **Bufs:** Προσωρινές ενισχύσεις.
- **Spawn rate:** Ρυθμός εμφάνισης εχθρών.

- **Shooter:** είδος παιχνιδιού βολών (συνήθως σφαίρες).
- **Kill count:** Αριθμός εχθρών που έχουν εξουδετερωθεί.

3. Ζόμπι και Χαρακτηριστικά

- **Old Crone, Nerd, Greybeard, Brute, Torch:** Τύποι ζόμπι με ξεχωριστές ιδιότητες.
- **Speed:** Ταχύτητα κίνησης.
- **Damage:** Ζημιά που προκαλεί το ζόμπι.
- **Special Abilities:** Ειδικές ικανότητες (π.χ., ρίψη health pack από το Torch Zombie).

4. Εργαλεία και Χαρακτηριστικά Animation

- **Animation rigging:** Σύστημα δημιουργίας σκελετών για χαρακτήρες.
- **Two Bone IK Constraint:** Μηχανισμός ελέγχου κίνησης χεριών ή ποδιών.
- **Root Motion:** Έλεγχος κίνησης μέσω animation.
- **Blending:** Ομαλές μεταβάσεις μεταξύ animations.
- **layers:** Στρώσεις animation για μεγαλύτερη ευελιξία.
- **Avatar Masks:** Εφαρμογή animation σε συγκεκριμένα μέρη ενός χαρακτήρα.
- **Triggers:** Παράμετροι ενεργοποίησης μεταβάσεων animation.

5. Ορολογία Προγραμματισμού και Τεχνικές

- **NavMeshgent:** Εργαλείο πλοήγησης τεχνητής νοημοσύνης (AI navigation).
- **Serialized Fields:** Μεταβλητές που ρυθμίζονται μέσω του Unity Inspector.
- **Refocous Intervaal:** Διάστημα χρόνου ανανέωσης στόχου.
- **ZombieController:** Script που διαχειρίζεται τη συμπεριφορά των ζόμπι.
- **MoveToTarget:** Μέθοδος που κατευθύνει τα ζόμπι προς τον παίκτη.
- **HandleBuffExpiration:** Μέθοδος για τον έλεγχο λήξης buffs.
- **StartMoving / StopMoving:** Μέθοδοι που ελέγχουν την έναρξη και τη διακοπή της κίνησης.

6. Σκηνές και Δομή Παιχνιδιού

- **StartScene:** Αρχική σκηνή του παιχνιδιού.
- **MainScene:** Κεντρική σκηνή παιχνιδιού.
- **Pause Menu:** Μενού παύσης παιχνιδιού.
- **DefeatScene:** Σκηνή που εμφανίζεται όταν ο παίκτης χάνει.
- **SceneManager.LoadScene:** Μέθοδος φόρτωσης σκηνής.
- **SceneManager.LoadSceneAsync:** Μέθοδος ασύγχρονης φόρτωσης σκηνής.

7. Συστήματα και Μηχανισμοί

- **Health Mechanics:** Σύστημα υγείας για τον παίκτη και τους εχθρούς.
- **Animation Events:** Εκτέλεση λειτουργιών κατά τη διάρκεια animation.
- **Dynamic Adjustment:** Προσαρμογή κινήσεων χαρακτήρα σε πραγματικό χρόνο.
- **Constraints:** Περιορισμοί σε κινήσεις χαρακτήρων.

8. Εργαλεία και Υποδομές

- **Mixamo:** Πλατφόρμα δημιουργίας animations και 3D μοντέλων.
- **ScreenshotManager:** Script για διαχείριση στιγμιότυπων.
- **Timer:** Script που παρακολουθεί τον χρόνο.

9. Τύποι Δεδομένων και Μεταβλητές

- **Booleans:** Μεταβλητές τύπου αληθές/ψευδές.
- **Floats/Integers:** Μεταβλητές αριθμητικής μορφής.

- **Triggers:** Σημεία ενεργοποίησης μεταβάσεων.
- **Parameters:** Παράμετροι που ελέγχουν τη συμπεριφορά animation.

10 Συμπεράσματα

Η δημιουργία του *Survive The Zombies*, ενός survival/horde mode παιχνιδιού, αποτέλεσε μια συναρπαστική αλλά και απαιτητική εμπειρία. Κατά τη διάρκεια της πτυχιακής εργασίας, είχα την ευκαιρία να εμβαθύνω στον κόσμο του game development, να ανακαλύψω τις προκλήσεις του και να δοκιμάσω τις δυνατότητές μου σε ένα ευρύ φάσμα τομέων, από το σχεδιασμό χαρακτήρων και animations, μέχρι τον προγραμματισμό και την ενσωμάτωση πολύπλοκων μηχανισμών παιχνιδιού.

Αρχικά, οι στόχοι της εργασίας ήταν η κατανόηση των βασικών αρχών δημιουργίας παιχνιδιών και η εξοικείωση με εργαλεία όπως το Unity και η γλώσσα προγραμματισμού C#. Παρόλο που υπήρχαν εισαγωγικές γνώσεις σε κάποιους τομείς, όπως το 3D modelling και το scripting, οι απαιτήσεις του έργου με ώθησαν να εξερευνήσω και νέους τομείς, όπως το rigging και το animation, αναπτύσσοντας έτσι μια ολοκληρωμένη εικόνα για τη διαδικασία δημιουργίας παιχνιδιών.

Η μεγαλύτερη πρόκληση ήταν η αντιμετώπιση τεχνικών προβλημάτων, όπως οι απαιτήσεις απόδοσης των υπολογιστικών συστημάτων. Παρά τις δυσκολίες, αυτές οι εμπειρίες με βοήθησαν να κατανοήσω τη σημασία της οργάνωσης, της διαχείρισης χρόνου και της συνεργασίας για την επιτυχή ολοκλήρωση ενός έργου. Επιπλέον, η επαναλαμβανόμενη διαδικασία πειραματισμού (trial and error) υπήρξε καθοριστική για την επίλυση των προβλημάτων και την τελική βελτίωση του παιχνιδιού. Η ανάπτυξη του παιχνιδιού περιλάμβανε ποικίλες προκλήσεις. Αρχικά, η εμβάθυνση σε τεχνικές όπως το 3D modeling, το animation και το rigging, ήταν ιδιαίτερα απαιτητική. Παρόλο που είχα μια βασική εξοικείωση με αυτές τις διαδικασίες, η πτυχιακή εργασία με ώθησε να αναπτύξω δεξιότητες σε νέες περιοχές και να εμβαθύνω σε εργαλεία όπως το Animator και τις τεχνικές physics-based motion.

Ένα από τα πιο δημιουργικά κομμάτια της εργασίας ήταν ο σχεδιασμός των μηχανισμών του παιχνιδιού, όπως το σύστημα XP, οι ενισχύσεις (buffs) και τα επίπεδα δυσκολίας, καθώς και η προσαρμογή των animations μέσω scripting. Το να βλέπω τις ιδέες μου να ζωντανεύουν στην οθόνη και να λειτουργούν όπως ακριβώς τις είχα φανταστεί, αποτέλεσε μια μοναδική επιβράβευση και ενίσχυσε την αποφασιστικότητά μου να συνεχίσω στον τομέα του game development.

Συνοψίζοντας, το *Survive The Zombies* ήταν η πρώτη μου ολοκληρωμένη προσπάθεια στον τομέα της δημιουργίας παιχνιδιών και μια εξαιρετική βάση για μελλοντικά projects. Μέσα από τη διαδικασία αυτή έμαθα πώς να ξεπερνάω εμπόδια, να συνεργάζομαι αποτελεσματικά και να εξελίσσω τις δεξιότητές μου. Η εμπειρία αυτή μου έδωσε όχι μόνο τις τεχνικές γνώσεις που απαιτούνται για την ανάπτυξη ενός παιχνιδιού αλλά και την έμπνευση να συνεχίσω να δημιουργώ και να βελτιώνομαι. Ανυπομονώ για τα επόμενα βήματα και τα νέα δημιουργικά ταξίδια στον κόσμο του game development.

11 Βιβλιογραφία

- [1] Unity game engine <https://unity.com/>
- [2] Mixamo <https://www.mixamo.com/#/>
- [3] Unity Asset store <https://assetstore.unity.com/>
- [4] UnityLearn , official unity tutorials
<https://learn.unity.com/tutorials?k=%5B%22lang%3Aen%22%2C%22sl%3Abeginner%22%5D&ob=starts>
- [5] Sung, Kelvin και Gregory Smith. Basic Math for Game Development with Unity 3D: A Beginner's Guide to Mathematical Foundations. 2019. Εκδότης: CRC Press.
- [6] Nystrom, Robert. *Game Programming Patterns*. Αυτοέκδοση, 2014. Διαθέσιμο στη διεύθυνση: <https://gameprogrammingpatterns.com>.